



MSX: Lightweight Block Ciphers for Microcontrollers with High-assurance against Differential and Linear Attacks

Kazuhiko Minematsu¹, Tomoyasu Suzaki², Mostafizar Rahman³,
Sahiba Suryawanshi⁴ and Takanori Isobe⁵

¹ NEC Corporation, Kawasaki, Japan

² NEC Solution Innovator, Hokuriku, Japan

³ Kyoto University, Kyoto, Japan

⁴ University of Hyogo, Hyogo, Japan

⁵ The University of Osaka, Osaka, Japan

Abstract. We present MSX, a new family of 64/128-bit block ciphers. It aims to provide fast execution on microcontrollers and comes with a highly reliable argument on the resistance against basic differential/linear attacks, backed by the classical differential/linear probability analysis on Feistel ciphers and Vaudenay’s decorrelation theory. MSX are classical (generalized) Feistel ciphers with a round function. Similar to many existing ARX ciphers, its round function uses arithmetic operations and does not have an S-box. A unique feature of MSX is its use of 32-bit integer multiplication, which enables proving an ideally strong differential/linear property by design. It could be interpreted as an application of Vaudenay’s decorrelation theory. We provide a detailed security analysis on attacks beyond differential and linear ones and conduct a benchmark on a range of popular microcontrollers with a comparison to Speck, a top performer on microcontrollers. The results show MSX’s good performance on 32-bit microcontrollers, maintaining a sufficiently large security margin. MSX aims at security under single key, and related-key security is not the focus.

Keywords: Lightweight block cipher · Microcontroller · Universal Hash Function · Integer Multiplication · Decorrelation Theory

1 Introduction

The lightweight block cipher has been one of the central topics in symmetric-key primitive designs over the past decades. If one aims to design a software-oriented cipher, particularly for microcontrollers, mainstream approaches can be broadly categorized into two groups. The first group is Addition-Rotation-XOR (ARX) or its variants, and the second is S-box-based ciphers, which are traditionally hardware-efficient but enhanced with some software-friendly features (*e.g.*, nibble/byte-wise operation) or optimized implementation techniques such as bitslicing or fixslicing. The former category is straightforward to implement on software by nature and yields many popular block ciphers such as XTEA [WN95], Speck [BSS⁺15], and LEA [HLK⁺14] (A Korean Standard). The performance of ARX ciphers in microcontrollers is generally excellent. At the same time, its security evaluation is much more challenging than traditional S-box-based designs [ALLW14, BRV14, BV14, Din14, LFW⁺16, HW20]. The

E-mail: k-minematsu@nec.com (Kazuhiko Minematsu), tomoyasu.suzaki@nec.com (Tomoyasu Suzaki), mrahman454@gmail.com (Mostafizar Rahman), sahibas@iitbhilai.ac.in (Sahiba Suryawanshi), takanori.isobe@ist.osaka-u.ac.jp (Takanori Isobe)



second category has also been actively studied and resulted in a lineup of well-studied ciphers, *e.g.*, nibble-oriented designs Twine [SMMK13] and LBlock [WZ11] or bitslice-friendly designs Noekeon [DPA00], Rectangle [ZBL⁺14], SEA [SPGQ06], RoadRunner [BS15] and those based on LS-design [GLSV15]. In particular, GIFT [BPP⁺17] with fixslicing [ANP20] runs exceptionally fast on microcontrollers despite its hardware-oriented design of GIFT. This second (S-box-based) category enables more precise security evaluations, while there remains some performance gap on microcontrollers from ARX ciphers. This situation suggests the following question:

Is there any design approach that enables rigorous security evaluation and performs competitively to state-of-the-art ARX block ciphers on microcontrollers?

This paper proposes an alternative approach to address the above question. Our approach is to build a large, strong round function utilizing arithmetic operations, which may sound similar to ARX S-boxes [DPU⁺16, BBdS⁺20]. However, the crucial difference is that we use *integer multiplication* together with integer addition and we prove a strong differential/linear property. In more detail, the non-linear layer of our 32-bit round function guarantees strong resistance against differential and linear attacks, as it implements a strong universal (SU) hash function [Tho15]. A round function being SU means that the two output values for any two distinct input values are independent and uniformly distributed when the keys are uniform, and it implies that the differential and linear probabilities are the smallest possible values since the output values are uniformly distributed. Considering resistance against beyond basic differential/linear attacks, the round function also has a simple bit rotation-based linear layer followed by the non-linear layer, maintaining the linear/differential probabilities mentioned above thanks to its invertibility. We compose this round function in the classical (generalized) Feistel structure. The resulting cipher consists of strong and large round functions with a simple word-wise structure, which significantly simplifies security analysis. Concretely, we propose a family of ciphers, dubbed MSX (for Multiply-Sum-XOR), consisting of a 64-bit block cipher MSX-64 and a 128-bit block cipher MSX-128. The former has a 128-bit key and the latter has a 128 or 256-bit key. We propose MSX as a family of lightweight block ciphers, where “lightweight” is a generic word widely used by numerous post-AES ciphers, both hardware- and software-oriented. Following this tradition, we define lightweight to mean designs aiming at performance advantage over classical ciphers, most notably AES, enabled by optimized or new mechanisms. Target performance measure and platform may vary, either speed or implementation size, on software or hardware. For MSX, the target is microcontrollers, aiming at fast operation with high assurance against differential/linear attacks (see below).

We remark that our approach itself is not new: Vaudenay proposed the decorrelation theory in his series of works [Vau98, Vau99b, Vau99a, Vau03]. Our proposals could be interpreted as ciphers built on a decorrelation module of order 2 [Vau03], also known as Decorrelated Fast Ciphers (DFCs). A specific instantiation of DFC was proposed for the AES competition [PV98] and received attention for its innovative approach. Variants/successors [GNNV01, CLLL00] were also proposed. However, attacks against DFC beyond what it guarantees were reported [KR99] and after the AES competition, this approach has been largely abandoned (few exceptions exist such as [Mac00, BF07]). Our purpose is to revisit this classical approach and show its usefulness in modern software-oriented cipher designs. For this purpose, we looked into a class of universal hash (UH) functions studied in the theoretical computer science community [Tho15], so-called “without primes” [Die96], and found that it is quite useful on microcontrollers. The main reason for fast operation is the use of integer multiplication of *half-size output* (*i.e.*, the lower half of $x \times y$, for $x, y \in \{0, \dots, 2^w - 1\}$ for some w), which is quite fast, often runs in a single cycle, on popular microcontrollers. Moreover, this UH function does not implement a field multiplication (unlike common UH functions in cryptography such as GHASH [MV04] or Poly1305 [Ber05]), which is beneficial on microcontrollers as modular reduction is costly on

embedded platforms even with division-less algorithms. Despite a huge amount of research in the cryptography community on fast UH functions for more than two decades, such as [HK97, BHK⁺99, CGS17, GAED23], to the best of our knowledge, the UH function we adopt has never been used in a concrete cryptographic primitive.

We conducted a benchmark of **MSX** on 32-bit AVR/ARM microcontrollers and compared ours with **Speck** as one of the top performers on these platforms. While **Speck** still runs faster, the performance of **MSX** is also good, within a loss factor of around 1.5 in most cases. We also benchmarked AES on the same platforms using the state-of-the-art fixslicing [AP21] and confirmed superior performance of 128-bit block versions of **MSX**.

For the security analysis, we use MILP modeling to estimate the bounds corresponding to differential and linear trails, which are useful for estimating security against attacks beyond basic differential/linear attacks. Existing models are applied for the modulo addition operation [FWG⁺16, SHY16], while a new method is derived to estimate the upper bound on the probability for modulo multiplication. These models are combined to calculate upper bounds on the probabilities of differential and linear trails. Additionally, extensive analysis is conducted on **MSX** to evaluate its resistance against algebraic, rotational, boomerang, and differential-linear attacks, among others.

Our scope is black-box security, and side-channel attacks are not the focus of this study (a future topic). Nevertheless, our round function is suitable for constant-time operation for a wide range of microcontrollers (See Sect. 4.1). The implementations and the test vectors are available at <https://github.com/mrahman454/Block-Cipher-MSX.git/>.

1.1 Related work

ARX is extensively employed by cryptographic primitives other than block cipher, such as stream ciphers (ChaCha20 [Ber08]), MACs/PRFs (Chaskey [MMV⁺14] or SipHash [AB12]), and hash functions (Blake [AMPH14] or Skein [FLS⁺10]). NIST Lightweight Cryptography (LwC) project aims at standardizing lightweight authenticated encryption. Among the finalists of LwC, there are Sponge constructions either based on an ARX permutation (Sparkle [BBS⁺19]) or bitslice-oriented S-box-based permutation (the winner, Ascon [DEMS19]). While the performance figures of these finalists on microcontrollers are excellent, a fair performance comparison with our proposals is difficult: for example, the internal 320-bit permutation (Ascon-p) in Ascon is exclusively used in the sponge construction and not intended to be used as a component in a block cipher, say, an Even-Mansour cipher. Existence of distinguishers against Ascon-p, *e.g.*, [GPT21], also complicates fair comparison. Some recent designs took a mixed approach, namely using a large (*e.g.*, 32 or 64-bit) S-box built on ARX and composing it in traditional Feistel/SPN structure [DPU⁺16, BBS⁺20]. They come with an advanced security evaluation against differential/linear cryptanalysis [DPU⁺16]. Some classical ciphers such as IDEA [LM91] and RC6 [RRY98] use multiplication as a source of non-linearity and diffusion. MOE [BFL⁺21] uses multiplication by 3 modulo 2^{128} . Noilhan [Noi99] studied performance optimization of decorrelation module.

2 Preliminaries

For integers $0 \leq i < j$, let $[i..j] := \{i, i+1, \dots, j\}$. Let $\{0, 1\}^*$ be the set of all finite bit strings. For $X \in \{0, 1\}^*$, $|X|$ is its length in bits. The empty string is denoted by ε and $|\varepsilon| = 0$. We may use $\mathbb{F}_2^n$ and $\{0, 1\}^n$ interchangeably. Let $\{0, 1\}^{\leq b}$ denote $\bigcup_{i=0,1,\dots,b} \{0, 1\}^i$, where $\{0, 1\}^0 = \{\varepsilon\}$. For two bit-strings X and Y , $X \parallel Y$ is their concatenation. We also write this as XY if it is clear from the context. Let 0^i be the string of i zero bits; for instance, we write 10^i for $1 \parallel 0^i$. For $X \in \{0, 1\}^*$ with $|X| \geq i$, $\text{msb}_i(X)$ is the first (left) i bits of X , and $\text{lsb}_i(X)$ is the least-significant (most-significant) i bits of X . If X is uniformly chosen

from the set $\mathcal{X}$, we write $X \stackrel{s}{\leftarrow} \mathcal{X}$. For variables $X_1, X_2, \dots, X_s$ for some $s \geq 1$, $i < j \leq s$, $X_j \parallel \dots \parallel i$ denotes $X_j \parallel X_{j-1} \parallel \dots \parallel X_i$. Moreover, for $i_1, \dots, i_s \in \{1, \dots, s\}$, $X_{(i_1, i_2, \dots, i_s)}$ and $X_{i_1} \parallel i_2 \parallel \dots \parallel i_s$ denote $(X_{i_1}, X_{i_2}, \dots, X_{i_s})$ and $X_{i_1} \parallel X_{i_2} \parallel \dots \parallel X_{i_s}$.

For two integers $X, Y \in [0..2^w - 1]$, let $X \boxplus_w Y$ denote the integer addition modulo 2^w . Let $X \times_w Y$ denote the integer multiplication modulo 2^{2w} , a function: $\{0, 1\}^w \times \{0, 1\}^w \rightarrow \{0, 1\}^{2w}$, and let $X \boxtimes_w Y$ denote the last w bits (*i.e.*, the lower half) of $X \times_w Y$. For $X \in \{0, 1\}^*$ and $i \leq |X|$, $\text{ROL}(X, i)$ (resp. $\text{ROR}(X, i)$) denotes the i -bit left (resp. right) rotate shift of X . We call $\times_w$ *full* multiplication and $\boxtimes_w$ *half* multiplication over w bits. If w is obvious from the context we may drop it. For any bit string, we assume a natural encoding to non-negative integers. For example, a bit string $X = (1011)$ is interpreted as integer 11, and $\text{msb}_2(X) = (10)$, interpreted as 2, and $\text{lsb}_2(X) = (11)$, interpreted as 3.

Universal hash functions.

Definition 1. If $H : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is ϵ -almost strongly universal (ϵ -ASU),

$$\Pr_{K \stackrel{s}{\leftarrow} \mathcal{K}} [H(K, X_0) = Y_1, H(K, X_1) = Y_2] \leq \epsilon$$

for any distinct $X_0, X_1 \in \mathcal{X}$ and any $Y_1, Y_2 \in \mathcal{Y}$. If the above holds for $\epsilon = \frac{1}{|\mathcal{Y}|^2}$, which is the minimum, we say H is strongly universal (SU) also known as pairwise independent.

Similarly, if $\mathcal{Y} = \{0, 1\}^m$ for some $m \geq 1$ and $\Pr_{K \stackrel{s}{\leftarrow} \mathcal{K}} [H(K, X_0) \oplus H(K, X_1) = \delta] \leq \epsilon$ holds any distinct $X_0, X_1 \in \mathcal{X}$ and any $\delta \in \{0, 1\}^m$, H is said to be ϵ -almost XOR universal (ϵ -AXU). If the above holds only for $\delta = 0$, H is said to be ϵ -almost universal (ϵ -AU).

Clearly, ϵ -ASU implies ϵ -AXU (if $\mathcal{Y} = \{0, 1\}^m$) and ϵ -AXU implies ϵ -AU. The converses do not hold.

Differential/Linear probability. Let $E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$ be a keyed permutation on $\mathcal{M}$ with key $K \in \mathcal{K}$. Assuming uniform key $K \stackrel{s}{\leftarrow} \mathcal{K}$, the Maximum Expected Differential Probability (MEDP) of E is defined as

$$\max_{a, b \neq 0^n} \Pr_{K \stackrel{s}{\leftarrow} \mathcal{K}, M \stackrel{s}{\leftarrow} \mathcal{M}} [E_K(M) \oplus E_K(M \oplus a) = b].$$

Similarly, the Maximum Expected Linear Probability (MELP) of E (with uniform key) is defined as

$$\max_{a, b \neq 0^n} \left(2 \cdot \Pr_{K \stackrel{s}{\leftarrow} \mathcal{K}, M \stackrel{s}{\leftarrow} \mathcal{M}} [a \odot M = b \odot E_K(M)] - 1 \right)^2,$$

where $A \odot B$ denotes bit-wise inner product.

3 Specification of MSX

In this section, we specify MSX, a family of block ciphers consisting of MSX-64 and MSX-128. MSX-64 has a 64-bit block and a 128-bit key (Algorithm 1) and MSX-128 has a 128-bit block and a 128 or 256-bit key (Algorithm 2). We write MSX-128-128 and MSX-128-256 to denote the 128-bit and 256-bit key versions. The global structure of MSX is classical. MSX-64 is a (2-branch) balanced Feistel cipher, and MSX-128 is a Type-2 4-branch generalized Feistel cipher, both working on 32-bit branches. MSX-64 has 14 rounds. MSX-128-128 and MSX-128-256 have 18 rounds. The core of the ciphers is a keyed function that maps 32-bit input to 32-bit output taking 192-bit key, which will be detailed below. The round keys are derived from a dedicated key schedule for each member of MSX shown by Algorithms 3, 4 and 5. See also Figure 1.

Algorithm 1: Data processing of MSX-64

1: function MSX-64.ENC(K, M) 2: $(X_1, X_0) \xleftarrow{32} M$ 3: $RK \leftarrow \text{KS}_{64,128}(K)$ 4: for $i = 1$ to 14 do 5: $X_0 \leftarrow X_0 \oplus \text{RFnc}(RK_i, RC_i, X_1)$ 6: $(X_0, X_1) \leftarrow (X_1, X_0)$ 7: return $C \leftarrow X_0 \parallel X_1$	1: function MSX-64.DEC(K, C) 2: $(X_1, X_0) \xleftarrow{32} C$ 3: $RK \leftarrow \text{KS}_{64,128}(K)$ 4: for $i = 14$ to 1 do 5: $X_0 \leftarrow X_0 \oplus \text{RFnc}(RK_i, RC_i, X_1)$ 6: $(X_0, X_1) \leftarrow (X_1, X_0)$ 7: return $M \leftarrow X_0 \parallel X_1$
--	--

Algorithm 2: Data processing of MSX-128- κ , $\kappa \in \{128, 256\}$

1: function MSX-128- κ .ENC(K, M) 2: $(X_3, X_2, X_1, X_0) \xleftarrow{32} M$ 3: $RK \leftarrow \text{KS}_{128,\kappa}(K)$ 4: for $i = 1$ to 18 do 5: $X_0 \leftarrow X_0 \oplus \text{RFnc}(RK_{i,0}, RC_i, X_1)$ 6: $X_2 \leftarrow X_2 \oplus \text{RFnc}(RK_{i,1}, RC_i, X_3)$ 7: $X_{(0,3,2,1)} \leftarrow X_{(3,2,1,0)}$ 8: return $C \leftarrow X_0 \parallel X_3 \parallel X_2 \parallel X_1$	1: function MSX-128- κ .DEC(K, C) 2: $(X_3, X_2, X_1, X_0) \xleftarrow{32} C$ 3: $RK \leftarrow \text{KS}_{128,\kappa}(K)$ 4: for $i = 18$ to 1 do 5: $X_0 \leftarrow X_0 \oplus \text{RFnc}(RK_{i,0}, RC_i, X_1)$ 6: $X_2 \leftarrow X_2 \oplus \text{RFnc}(RK_{i,1}, RC_i, X_3)$ 7: $X_{(2,1,0,3)} \leftarrow X_{(3,2,1,0)}$ 8: return $M \leftarrow X_2 \parallel X_1 \parallel X_0 \parallel X_3$
---	---

3.1 Round function

The round function consists of non-linear and linear layers (Figure 2). The non-linear layer is $\text{RFnc} : \mathcal{K}_{\text{RFnc}} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{32}$, where $\mathcal{K}_{\text{RFnc}} = (\{0, 1\}^{32})^6$ denotes the key space. Let $\text{hRFnc} : \mathcal{K}_{\text{hRFnc}} \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{16}$, where $\mathcal{K}_{\text{hRFnc}} = (\{0, 1\}^{32})^3$, be a component function of RFnc such that

$$\text{hRFnc}((K_3, K_2, K_1), X) = \text{msb}_{16}(((K_1 \boxplus_{32} \text{msb}_{16}(X)) \boxtimes_{32} (K_2 \boxplus_{32} \text{lsb}_{16}(X))) \boxplus_{32} K_3).$$

RFnc consists of two independently-keyed evaluations of hRFnc on the input;

$$\begin{aligned} \text{RFnc}((K_{(6,\dots,1)}), RC_i, X) &:= \text{mix}(\text{hRFnc}((K_{(3,2,1)}), X) \parallel (\text{hRFnc}((K_{(6,5,4)}), X) \oplus RC_i)), \\ \text{mix}(X) &:= X \oplus \text{ROL}(X, 13) \oplus \text{ROL}(X, 21). \end{aligned}$$

The linear layer inserts round constants RC_i and mixes the 16-bit output data from each hRFnc . RC_1 is 41421 (this value is the fractional part of $\sqrt{2}$) and RC_{i+1} is $RC_i + 1$. The mixing process is described $X \oplus \text{ROL}(X, 13) \oplus \text{ROL}(X, 21)$. One can easily verify that mix is a permutation over $\{0, 1\}^{32}$.

3.2 Key schedule

Let $\text{KS}_{n,k}$ denote the key scheduling function for n -bit block and k -bit key, where $(n, k) \in \{(64, 128), (128, 128), (128, 256)\}$. $\text{KS}_{64,128}$ have 12 states and $\text{KS}_{128,k}$ have 24 states. Each state has a 32-bit value. The whole state has round keys for two rounds. The state is initialized by taking an XOR of the master key and a constant (Table 1). All variants are simple, similar to word-oriented linear feedback shift registers (LFSR). To update the states, we apply an arithmetic addition of the source state S_i to multiple target states. After this step, each target state is first cyclically shifted before being added. Algorithms 3, 4, and 5 define $\text{KS}_{64,128}$, $\text{KS}_{128,128}$, $\text{KS}_{128,256}$, respectively. Figure 3 depicts $\text{KS}_{64,128}$.

4 Design Rationale

The components of MSX are not new: UH function as a round function (see below), (generalized) Feistel structure, and simple LFSR-like key schedule. We stress that this

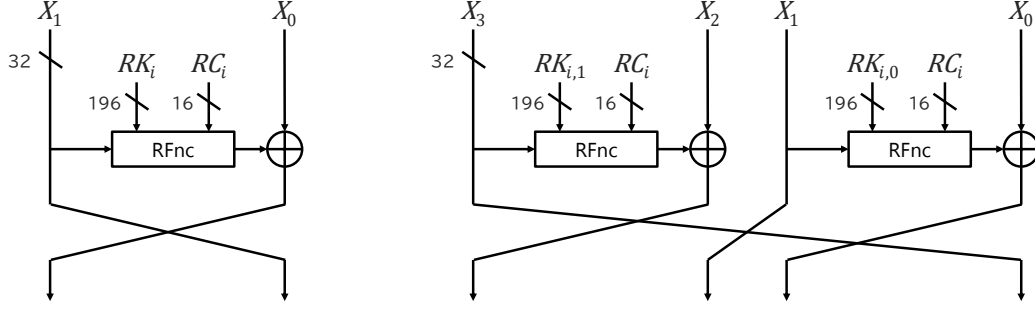


Figure 1: A round of MSX-64 (Left) and MSX-128 (Right).

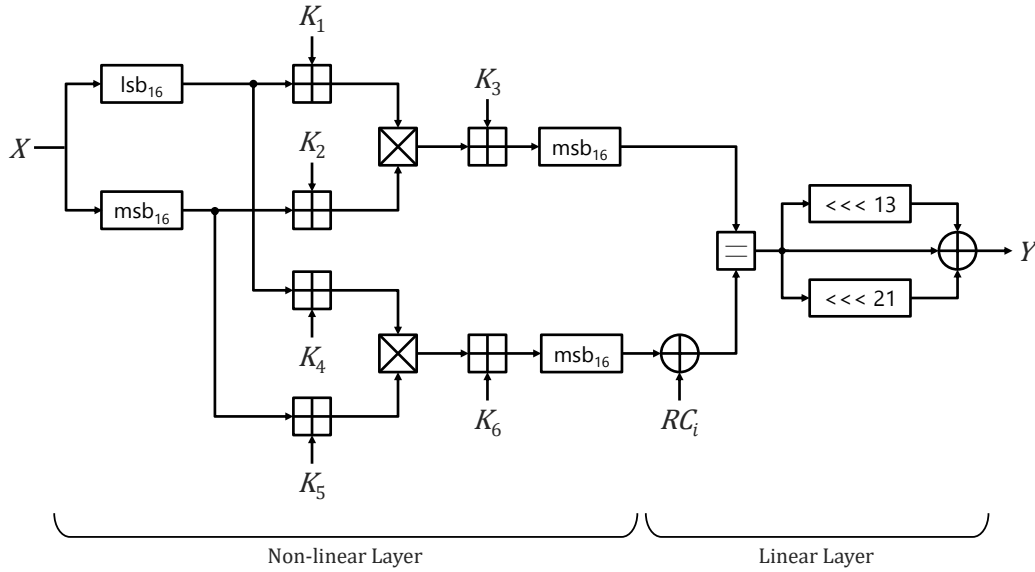


Figure 2: Round function of MSX (RFnc), where $|X| = |Y| = 32$ and the key is $K = (K_6, \dots, K_1)$, $|K_i| = 32$. All lines are 32 bits, where msb_{16} (lsb_{16}) extracts the upper (lower) 16 bits and prepends 16 zeros (*i.e.*, extracts as an integer $\in \{0, \dots, 2^{16} - 1\}$ represented in 32 bits). The symbol $\boxtimes$ denotes a 32-bit half multiplication.

choice is intentional: use of UH function ensures strong pairwise independence, which gives immunity against differential/linear attack with strong confidence, when composed in a (generalized) Feistel structure. Our purpose is to show that such a straightforward (yet overlooked) combination can yield efficient ciphers on microcontrollers, where the most of top performers are dominated by ARX ciphers, such as Speck.

As mentioned in Section 1, the basic idea of MSX dates back to Vaudenay [Vau03]. The crucial difference is in its instantiation. We adopt a strongly UH function utilizing (half) integer multiplications for the non-linear layer of RFnc. Throup [Tho15] presents a collection of software-oriented UH functions, and among them, those called “without primes” [Die96] are particularly interesting for their simplicity, despite little attention in the cryptography community. Moreover, we observed that a large population of microcontrollers perform 32-bit half integer multiplication within a single cycle, hence such UH functions will be very efficient and easy to be implemented in a constant-time manner. We present more detail below.

Algorithm 3: Key scheduling of MSX-64

```

1: function KS64,128(K)
2:    $(K_3, K_2, K_1, K_0) \stackrel{32}{\leftarrow} K$ ,
3:    $S_0 \leftarrow (K_0 \boxplus_{32} K_1 \boxplus_{32} K_3) \oplus KC_0$ ,
      $S_i \leftarrow K_0 \oplus KC_i$  ( $i = 2, 4, 7$ ),       $S_i \leftarrow K_1 \oplus KC_i$  ( $i = 5, 9$ )
      $S_i \leftarrow K_2 \oplus KC_i$  ( $i = 1, 8, 10$ ),     $S_i \leftarrow K_3 \oplus KC_i$  ( $i = 3, 6, 11$ )
4:   for  $i = 0$  to 5 do
5:      $RK_{2i+1} \leftarrow S_5 \parallel \dots \parallel 0$ ,    $RK_{2i+2} \leftarrow S_{11} \parallel \dots \parallel 6$ 
6:      $S_{1+i} \leftarrow \text{ROL}(S_{1+i}, 1) \boxplus_{32} S_i$ ,    $S_{3+i} \leftarrow \text{ROL}(S_{3+i}, 2) \boxplus_{32} S_i$ ,
      $S_{5+i} \leftarrow \text{ROL}(S_{5+i}, 3) \boxplus_{32} S_i$ ,    $S_{(7+i) \bmod 12} \leftarrow \text{ROL}(S_{(7+i) \bmod 12}, 4) \boxplus_{32} S_i$ ,
      $S_{(10+i) \bmod 12} \leftarrow \text{ROL}(S_{(10+i) \bmod 12}, 5) \boxplus_{32} S_i$ 
7:    $RK_{13} \leftarrow S_5 \parallel \dots \parallel 0$ ,    $RK_{14} \leftarrow S_{11} \parallel \dots \parallel 6$ 
   return  $RK \leftarrow (RK_{14}, \dots, RK_1)$ 

```

Algorithm 4: Key scheduling of MSX-128-128

```

1: function KS128,128(K)
2:    $(K_3, K_2, K_1, K_0) \stackrel{32}{\leftarrow} K$ 
3:    $S_0 \leftarrow (K_0 \boxplus_{32} K_2 \boxplus_{32} K_3) \oplus KC_0$ ,
      $S_i \leftarrow K_0 \oplus KC_i$  ( $i = 2, 4, 9, 12, 17, 19$ ),    $S_i \leftarrow K_1 \oplus KC_i$  ( $i = 1, 8, 10, 15, 18, 23$ ),
      $S_i \leftarrow K_2 \oplus KC_i$  ( $i = 5, 7, 14, 16, 21$ ),    $S_i \leftarrow K_3 \oplus KC_i$  ( $i = 3, 6, 11, 13, 20, 22$ )
4:   for  $i = 0$  to 7 do
5:      $RK_{2i+1,0} \leftarrow S_5 \parallel \dots \parallel 0$ ,       $RK_{2i+1,1} \leftarrow S_{11} \parallel \dots \parallel 6$ ,
      $RK_{2i+2,0} \leftarrow S_{17} \parallel \dots \parallel 12$ ,     $RK_{2i+2,1} \leftarrow S_{23} \parallel \dots \parallel 18$ 
6:      $S_{1+i} \leftarrow \text{ROL}(S_{1+i}, 1) \boxplus_{32} S_i$ ,       $S_{3+i} \leftarrow \text{ROL}(S_{3+i}, 2) \boxplus_{32} S_i$ ,
      $S_{6+i} \leftarrow \text{ROL}(S_{6+i}, 3) \boxplus_{32} S_i$ ,       $S_{9+i} \leftarrow \text{ROL}(S_{9+i}, 4) \boxplus_{32} S_i$ ,
      $S_{12+i} \leftarrow \text{ROL}(S_{12+i}, 5) \boxplus_{32} S_i$ ,     $S_{14+i} \leftarrow \text{ROL}(S_{14+i}, 6) \boxplus_{32} S_i$ ,
      $S_{(17+i) \bmod 24} \leftarrow \text{ROL}(S_{(17+i) \bmod 24}, 7) \boxplus_{32} S_i$ ,
      $S_{(20+i) \bmod 24} \leftarrow \text{ROL}(S_{(20+i) \bmod 24}, 8) \boxplus_{32} S_i$ ,
      $S_{(23+i) \bmod 24} \leftarrow \text{ROL}(S_{(23+i) \bmod 24}, 9) \boxplus_{32} S_i$ 
7:    $RK_{17,0} \leftarrow S_5 \parallel \dots \parallel 0$ ,       $RK_{17,1} \leftarrow S_{11} \parallel \dots \parallel 6$ ,
      $RK_{18,0} \leftarrow S_{17} \parallel \dots \parallel 12$ ,     $RK_{18,1} \leftarrow S_{23} \parallel \dots \parallel 18$ 
   return  $RK \leftarrow (RK_{18,1}, RK_{18,0}, \dots, RK_{2,1}, RK_{2,0}, RK_{1,1}, RK_{1,0})$ 

```

4.1 Round function

hRFnc is a 32-bit version of Pair-Multiply-Shift (PMS) function introduced by Thorup [Tho15, Sect. 3.5] based on the ideas of Dietzfelbinger’s AU function [Die96] and Black *et al.* [BHK⁺99]. From [Tho15], hRFnc is SU over 16 bits. Since RFnc composes two independent hRFncs and the outputs of hRFnc are concatenated and given to a permutation mix, RFnc is SU over 32 bits (proof is straightforward). Thus, RFnc is *ideally* strong against differential and linear attacks. Although an SU function can also be realized by table look up, it introduces two drawbacks that (1) the input size is limited to small as table grows up exponentially and (2) it may introduce non-constant-time operations which has a risk of timing attacks. It is also possible to realize an (A)SU function of large input based on smaller S-boxes by nesting the structure, as employed by Misty1 [Mat97], however it is not quite efficient. Therefore, this small S-box-based approach deviates from our goal of designing highly efficient cipher on microcontrollers comparable to state-of-the-art ARX ciphers, while ensuring a high reliance on the resistance against linear/differential attacks.

PMS has similarities to MMH [HK97] and NH [BHK⁺99], two well-known UH functions in the cryptographic community, however, there are important differences. MMH uses

Algorithm 5: Key scheduling of MSX-128-256

```

1: function KS128,256( $K$ )
2:    $(K_7, \dots, K_1, K_0) \xleftarrow{32} K$ 
3:    $S_0 \leftarrow (K_1 \boxplus_{32} K_2 \boxplus_{32} K_4 \boxplus_{32} K_5) \oplus KC_0$ ,
      $S_1 \leftarrow (K_0 \boxplus_{32} K_3 \boxplus_{32} K_6 \boxplus_{32} K_7) \oplus KC_1$ ,
      $S_i \leftarrow K_0 \oplus KC_i (i = 2, 12, 18)$ ,    $S_i \leftarrow K_1 \oplus KC_i (i = 8, 13, 19)$ ,
      $S_i \leftarrow K_2 \oplus KC_i (i = 6, 14)$ ,      $S_i \leftarrow K_3 \oplus KC_i (i = 7, 20)$ ,
      $S_i \leftarrow K_4 \oplus KC_i (i = 5, 15, 21)$ ,  $S_i \leftarrow K_5 \oplus KC_i (i = 11, 16, 22)$ ,
      $S_i \leftarrow K_6 \oplus KC_i (i = 3, 9, 17)$ ,   $S_i \leftarrow K_7 \oplus KC_i (i = 4, 10, 23)$ 
4:   for  $i = 0$  to 7 do
5:      $RK_{2i+1,0} \leftarrow S_5 \parallel \dots \parallel 0$ ,    $RK_{2i+1,1} \leftarrow S_{11} \parallel \dots \parallel 6$ ,
      $RK_{2i+2,0} \leftarrow S_{17} \parallel \dots \parallel 12$ ,    $RK_{2i+2,1} \leftarrow S_{23} \parallel \dots \parallel 18$ 
6:      $S_{1+i} \leftarrow \text{ROL}(S_{1+i}, 1) \boxplus_{32} S_i$ ,    $S_{3+i} \leftarrow \text{ROL}(S_{3+i}, 2) \boxplus_{32} S_i$ ,
      $S_{6+i} \leftarrow \text{ROL}(S_{6+i}, 3) \boxplus_{32} S_i$ ,    $S_{9+i} \leftarrow \text{ROL}(S_{9+i}, 4) \boxplus_{32} S_i$ ,
      $S_{12+i} \leftarrow \text{ROL}(S_{12+i}, 5) \boxplus_{32} S_i$ ,   $S_{14+i} \leftarrow \text{ROL}(S_{14+i}, 6) \boxplus_{32} S_i$ ,
      $S_{(17+i) \bmod 24} \leftarrow \text{ROL}(S_{(17+i) \bmod 24}, 7) \boxplus_{32} S_i$ 
      $S_{(20+i) \bmod 24} \leftarrow \text{ROL}(S_{(20+i) \bmod 24}, 8) \boxplus_{32} S_i$ 
      $S_{(23+i) \bmod 24} \leftarrow \text{ROL}(S_{(23+i) \bmod 24}, 9) \boxplus_{32} S_i$ 
7:    $RK_{17,0} \leftarrow S_5 \parallel \dots \parallel 0$ ,    $RK_{17,1} \leftarrow S_{11} \parallel \dots \parallel 6$ ,
      $RK_{18,0} \leftarrow S_{17} \parallel \dots \parallel 12$ ,    $RK_{18,1} \leftarrow S_{23} \parallel \dots \parallel 18$ 
   return  $RK \leftarrow (RK_{18,1}, RK_{18,0}, \dots, RK_{2,1}, RK_{2,0}, RK_{1,1}, RK_{1,0})$ 

```

Table 1: The constant values of the key schedule, represented in ingeters.
 These values are the fractional part of $\sqrt{3}$.

KC_1	732050807	KC_9	515107230	KC_{17}	645454543
KC_2	568877293	KC_{10}	1130980195	KC_{18}	3960268375
KC_3	527446341	KC_{11}	2149511253	KC_{19}	2795036687
KC_4	505872366	KC_{12}	539907735	KC_{20}	563113322
KC_5	942805254	KC_{13}	244569516	KC_{21}	2690074390
KC_6	634010619	KC_{14}	920961429	KC_{22}	722202776
KC_7	1296924710	KC_{15}	2743527186	KC_{23}	833026909
KC_8	3826869025	KC_{16}	2947265473	KC_{24}	2035301852

multiplication over a prime field and proves AXU. NH uses a full integer multiplication: $\{0, 1\}^w \times \{0, 1\}^w \rightarrow \{0, 1\}^{2w}$ for w suitable to the target platform in addition to integer addition modulo 2^w and proves AU. Note that there is no generic way to convert an AU function into an SU function. An improvement of NH dedicated for a short output was proposed by Nguen and Roscoe [NR12], however it still uses a full multiplication. Round functions in DFC and its variants [PV98, GNNV01, CLLL00, Vau03] use various classes of UH functions including SU, however, they basically implement a polynomial hash function on a prime field followed by reduction with a prime close to 2^w (similar to MMH/NH). Although use of such primes could avoid division in reduction, it is still costly on microcontrollers. In contrast, PMS uses a $w = 32$ -bit half integer multiplication: $\{0, 1\}^w \times \{0, 1\}^w \rightarrow \{0, 1\}^w$ in addition to w -bit integer addition. Without relying on a prime field, it ensures SU.

The use of a half multiplier is beneficial, especially for microcontrollers. ARM Cortex M series have various multiplication instructions, but only the simplest, half multiplier (MUL) operates at one cycle for all the range of Cortex M3 and M4 processors. A full multiplier instruction exists on M3 and M4, but is not guaranteed to operate at one cycle or constant time. The same holds for Cortex M0/M0+ CPUs, where 32-bit half multiplier instruction (MULS) operates at one cycle for platforms adopting fast implementation for multiplication¹.

¹<https://developer.arm.com/documentation/ddi0484/c/Programmers-Model/Instruction-set-summary>

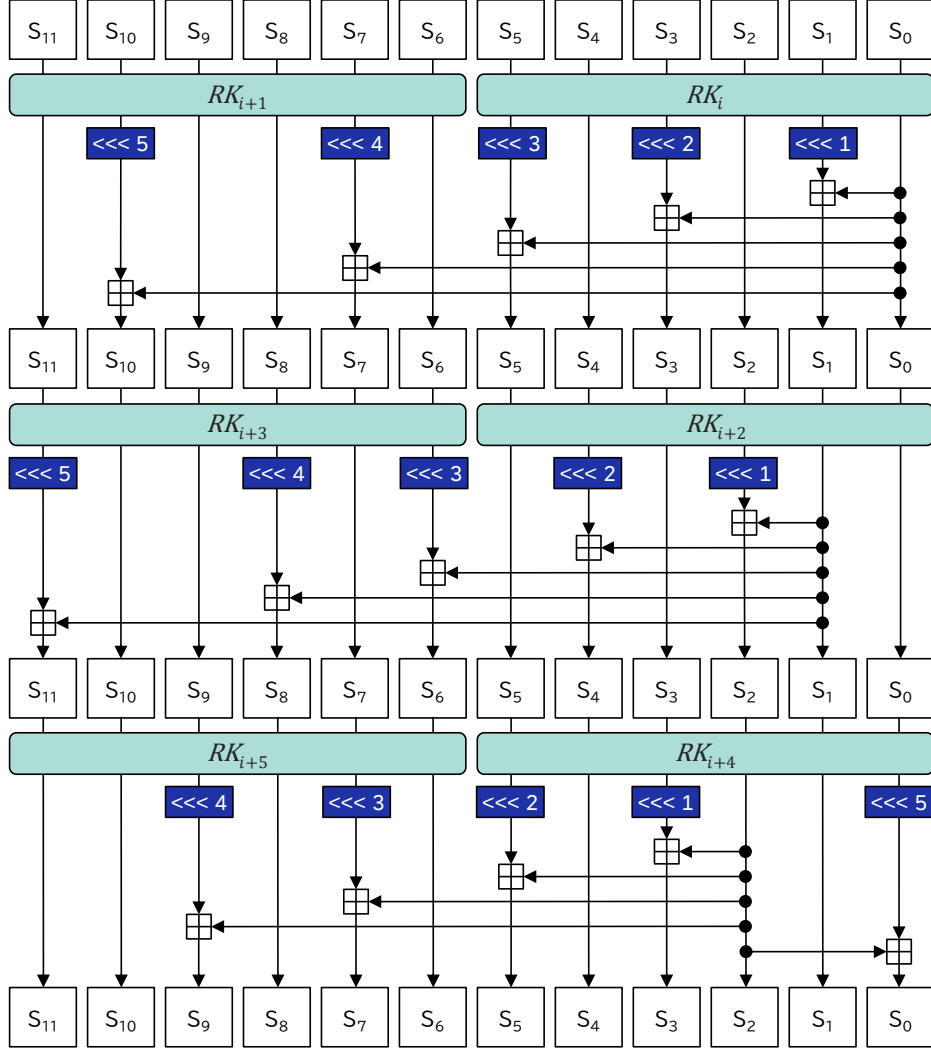


Figure 3: Key Scheduling of MSX-64.

See de Groot’s report [dG15] and Pornin’s analysis [Por18] for more information.

The non-linear layer of RFnc is a concatenation of two independently-keyed 16-bit hRFnc functions, thus potentially exhibits a slow diffusion. To gain faster diffusion, we introduce a linear layer mix . The rotation constants in mix are determined to give a balanced diffusion between 16-bit words, considering that the input to mix is a tuple of independent hRFnc ’s 16-bit outputs. If we write as $\text{mix}(X_L, X_R) = (Y_L, Y_R)$, each 16-bit word, 8 bits of Y_L are the sum of two bits among X_L and one bit among X_R , and the remaining 8 bits are the sum of two bits among X_R and one bit among X_L . The same holds for Y_R .

Let us extend and summarize our discussion on the efficiency/security trade-off between full and half multiplication. Use of 32-bit full multiplication enables classical field-multiplication-based universal hash functions as described above. As it directly operates on 32-bit words, it provides a 32-bit-to-32-bit function with strong diffusion over 32-bit word. In contrast, our hRFnc is a 32-bit-to-16-bit function, since the output truncation of the result of half multiplication is crucial for almost (strong) universality. Thus, we need two parallel instances of hRFnc to implement a 32-bit-to-32-bit function. In terms

of strong universality, which captures pairwise randomness of the output tuple (which is a stronger property than diffusion), both approaches yield almost equivalent security. A full multiplier is not always available on low-end 32-bit microprocessors. Hence, if we want a full multiplication-based round function (which would be a near 32-bit prime-field multiplication followed by reduction as DFC), we need a classical conversion method (e.g., Karatsuba) using a half multiplication to implement a full multiplication. Karatsuba requires 3 half multiplications and 6 additions/subtractions. Additional (a prime field to $\mathbb{Z}_{2^{32}}$) reduction and key addition will further increase the total cost. We also need to ensure the constant-time operation as a whole. This could be non-trivial. While our approach requires 2 half multiplications and 6 additions to implement the non-linear layer of 32-bit-to-32-bit ASU function **RFnc**. All the instructions are simple and easy to check their constant-time operation. ARM M3 and M4 have instructions for full multiplication (**smull** and **umull**), but on M3, they are not constant-time and it is not simple to determine the required cycle (See [Por18]). Hence it is not safe to use if we care about timing side-channel attacks. Considering the balanced performance over a range of 32-bit microcontrollers, we think our approach is relevant.

Round constants are inserted into a branch in the round function to thwart slide attacks and its variants. Since most operations in the round function are arithmetic, we chose XOR to absorb the constants for its incompatibility with binary field.

4.2 Key Scheduling

We design our key scheduling to avoid a shortcut in key guessing. In particular, since the output of **RFnc** is $\text{msb}_{16}(K_3 \oplus V_1) \parallel \text{msb}_{16}(K_6 \oplus V_2)$ for unknown V_1 and V_2 , the lower halves of K_3 and K_6 have small contribution to its output (see Fig 2). This makes output guessing reliable without knowing their lower halves. To force the full subkey guessing of K_3 and K_6 in the key guess procedure of an attack, we ensure that these two subkey materials of a specific **RFnc** call are also used by other subkeys (*i.e.*, K_1 , K_2 , K_4 , and K_5) of a neighbor **RFnc** that will be involved in key guessing. For $\text{KS}_{64,128}$ and $\text{KS}_{128,128}$, this could be achieved by ensuring that the first two rounds of **RFnc** contain all the key materials. For $\text{KS}_{128,256}$, due to the structure, multiple two-round paths for the key guessing among could be considered, and for each path we ensure the similar condition is satisfied (*i.e.*, the K_3 and K_6 of an **RFnc** will appear at K_1 , K_2 , K_4 , and K_5 of another call of **RFnc** on the path).

Initialization of the state is based on XORing each state with one word of the master key and a non-zero constant. However, only state S_0 (and S_1 in the case of $\text{KS}_{128,256}$) is initialized by adding multiple words and then XORing with a constant. This design ensures that all words of the master key influence every state with minimal updates. Specifically, in $\text{KS}_{64,128}$, all states are affected within 3 updates, and in $\text{KS}_{128,K}$, within 4 updates. Regarding state update, the number of taps and their positions will affect security. The distance between each updated state was set to less than 3. This is to ensure that among any three consecutive states — corresponding to the three round keys used in **hRFnc** — at least one of them is always updated.

4.3 Related/Weak keys

While we conducted preliminary analysis on weak/related keys of **MSX** and found no particular issues, we do not claim related-key security (following *e.g.* [BBI⁺20]) as it may have limited relevance in real use cases of lightweight ciphers. Due to the nature of multiplication, **hRFnc** has some unique behavior for specific key values. For example, the key tuple shown (K_1, K_2, K_3) in Table 2 will fix the MSB of the output. The number of such weak keys is not huge (262,114 among 2^{196}). Even if a state satisfying the conditions in Table 2 appears by chance, it would not persist due to the state update mechanism in

Table 2: Weak keys for hRFnc. K_1 and K_2 are represented in hexadecimal, while K_3 is in binary. 0^i denotes the sequence of i zeros, and $*^i$ denotes an arbitrary binary sequence of length i .

K_1	K_2	K_3
0xffff8000	0xffff8000	$*01^{15}*^{15}$
0xffff8000	0xffff8001	$*10^{15}*^{15}$
0xffff8001	0xffff8000	$*10^{15}*^{15}$
0xffff8001	0xffff8001	$*01^{15}*^{15}$

the key schedule. As shown in Algorithms 3, 4 and 5, among the 12 or 24 consecutive states, one of the three states used in hRFnc (i.e., the three round keys) is updated by a combination of its own rotation shift and arithmetic addition with another state. For example, to satisfy the equation $\text{ROL}(0xffff8000, i) + X = 0xffff8000$, the value of X must cancel out the shifted result. This condition is met by values such as $0x0007fff$ when $i = 1$, or $0x00ff7e01$ when $i = 9$. In these cases, at least the upper 8 bits of X must be zero. However, as shown in Table 2, there are no cases where the upper 8 bits are all zero, making such a condition infeasible in practice.

5 Security Analysis

5.1 Differential/Linear Probabilities

As described earlier, the core round function RFnc implements a 32-bit SU function. Combining this property with the seminal results of Knudsen and Nyberg [NK93, NK95, Nyb95], MSX-64 has MEDP $2 \cdot 2^{-64}$ and MELP $2 \cdot 2^{-64}$ just with four rounds, assuming uniform round subkeys (which is a widely-used assumption on evaluating security against statistical attacks). We note that if RFnc was invertible given a key, we could reduce the number of rounds to three [AO97]. For MSX-128, if RFnc *was* invertible, Kim *et al.* [KLS⁺08] shows that five rounds of Type-2 4-branch generalized Feistel cipher has sufficiently strong MEDP/MELP, $O(p^4)$ for a round function having probability/bias p .

Unfortunately, a provable MEDP/MELP bound is not known for the general case of Type-2 generalized Feistel ciphers with possibly non-injective round functions. Nonetheless, traditional active S-box-based analyses [TSL11, HNE22, HDE24] suggest that 10 rounds provide sufficient resistance against differential and linear attacks, provided that each round function is immune to differential and linear attacks.

Differential and linear properties of $\boxtimes_w$. Although MSX-64 has provable security against differential and linear attacks, to demonstrate its security against attacks beyond differential and linear attacks, or their combinations, we rely on characteristic bounds. Moreover, obtaining such bounds will also be helpful in proving the security of MSX-128. Suppose $x, y \in \mathbb{F}_2^w$, and $z = x \boxtimes_w y$, where $x = x_{w-1} \cdots x_0$, $y = y_{w-1} \cdots y_0$, and $z = z_{w-1} \cdots z_0$. Then, $z_i = s_i \oplus c_i$, where $s_i = \sum_{j=0}^i x_j y_{i-j}$ and c_i are the carry bits for $0 \leq i \leq w-1$. For the differential, assuming the carry bits are zero (or have no effect), the maximum probability of a differential trail (MPD) depends on the individual AND operations involved in each s_i . Based on this, an upper bound on the probability of a differential trail involving the $\boxtimes_w$ operation can be deduced. It is important to note that the effect of chained AND operations, as discussed in [SSS⁺20], must be considered. Furthermore, since each bit is related to multiple other bits, the effect of multiple chained AND operations, as analyzed in [JRS22], also needs to be taken into account. By following a similar approach, maximum absolute correlation (C^2) of a linear trail can also be estimated. Note that this approach does not offer a tight upper bound on the MPD

Table 3: MPD, C^2 of MSX-64 and MSX-128

Variant	MPD/ C^2	Rounds				
		2	4	6	8	10
MSX-64	MPD	2^{-36}	2^{-72}	2^{-106}	2^{-142}	-
	C^2	2^{-36}	2^{-72}	2^{-104}	2^{-138}	-
MSX-128	MPD	-	2^{-38}	2^{-68}	2^{-102}	2^{-132}
	C^2	-	2^{-38}	2^{-68}	2^{-102}	2^{-130}

or C^2 ; instead, it provides a loose bound that offers a reasonable justification for the security margin in terms of the total number of rounds. Combining the effect of differential properties of $\boxtimes_w$ along with the MILP models for the differential of the addition modulo operation [FWG⁺16, SHY16], the upper bound on the probability of the differential trails is estimated. Based on this analysis, 4-round, 6-round and 8-round of MSX-64 has MPD of 2^{-72} , 2^{-106} and 2^{-142} , and C^2 of 2^{-72} , 2^{-104} and 2^{-138} , respectively. The MPD and C^2 of both MSX-64 and MSX-128 are listed in Table 3.

5.2 Multiplicative Differentials

Multiplicative differentials introduced in [BCJW02] is a more general form of the attack introduced in [Fur01] which were used to attack **xmx** [MNSV97] and **MultiSwap** [Scr01]. We analyzed MSX against these kind of attacks and found that such attacks can be easily thwarted. Note that, due to the introduction of the operations **msb**₁₆ and **lsb**₁₆, attacks similar to the one mounted on **xmx** cannot be applied to MSX. These operations prevent the attacker from introducing specific differences which results in the multiplicative differential pairs $(x, x \oplus 1 \cdots 1)$, since the most significant 16 bits cannot be controlled.

Using the multiplicative differential pair $(x, 2x)$ (which was used to attack **MultiSwap**), a distinguisher can be constructed up to the first non-linear layer based on some constraints. Specifically, we observed that the operations **lsb**₁₆ and **msb**₁₆ preserve the multiplicative relation $(x, 2x)$, i.e., $2(\text{lsb}_{16}(x) \boxtimes y) = (\text{lsb}_{16}(2x) \boxtimes y)$ where $x, y \in \{0, 1\}^{32}$ and $x_{15} = 0$. This particular property can be exploited by setting the round keys $K_1 = K_3 = 0$, allowing a distinguisher to be constructed up to the first non-linear layer. However, the subsequent linear layer prevents the propagation of such properties. In general, the operations **lsb**₁₆ and **msb**₁₆, together with the linear layer, are effective in resisting such attacks. Hence, we believe that MSX is secure against multiplicative differentials.

5.3 Algebraic Analysis

To analyze the MSX cipher’s algebraic structure, we examine modulo addition and modulo multiplication separately, followed by their combined effect.

Modulo Addition. Consider two integers $X, Y \in [0, 2^n - 1]$, and let $X \boxplus_n Y$ denote integer addition modulo 2^n . To analyze its algebraic structure, we reinterpret the operation as an n -dimensional Boolean function with $2n$ input variables. Thus, $X \boxplus_n Y$ is represented by the output vector $\{f_n, f_{n-1}, \dots, f_1\}$, where $X = \{x_n, x_{n-1}, \dots, x_1\}$ and $Y = \{y_n, y_{n-1}, \dots, y_1\}$. Each f_i is given by:

$$\begin{aligned}
 f_i &= x_i \oplus y_i \oplus c_{i-1}, \quad \text{where } i \in \{n, \dots, 2, 1\} \text{ and,} \\
 c_i &= \begin{cases} 0, & i = 0, \\ x_i y_i \oplus c_{i-1} (x_i \oplus y_i), & \forall 1 \leq i \leq n. \end{cases}
 \end{aligned}$$

In the MSX cipher, the inputs to the modulo addition operation are the data X and the key K . Let $X = \{x_w, \dots, x_2, x_1\}$ and $K = \{k_w, \dots, k_1\}$ represent the binary

representations of the input and key, respectively. The modulo addition can be expressed as $f_i = x_i \oplus k_i \oplus c_{i-1}$ where $i \in \{w, \dots, 2, 1\}$. We observe that as we progress from LSB to MSB, the degree of each f_i increases by one. Specifically, $\deg(f_{i+1}) = \deg(f_i) + 1$ for all $1 \leq i < w$, as given in the following observation.

Observation 1. *After modulo addition, as we observe the output bits from LSB to MSB of the resultant modulo addition, where each bit is represented by f_i , the degree of each f_i increases by one. Specifically,*

$$\deg(f_{i+1}) = \deg(f_i) + 1, \quad \forall 1 \leq i \leq w.$$

We observe that as we progress from LSB to MSB, the degree of each f_i increases by one. Specifically,

$$\deg(f_{i+1}) = \deg(f_i) + 1, \quad \forall 1 \leq i < w,$$

which aligns with Observation 1.

Modulo Multiplication. To analyze the algebraic structure of $A \boxtimes_w B$, we reinterpret the operation as an w -dimensional Boolean function. The output vector $\{f_w, f_{w-1}, \dots, f_1\}$ is determined by $A = \{a_w, \dots, a_1\}$ and $B = \{b_w, \dots, b_1\}$, expressed as:

$$f_i = \bigoplus_{j=1}^i a_j b_{i-j+1}, \quad \text{for } 1 \leq i \leq w,$$

which satisfies the following results.

Lemma 1. *After modulo multiplication, as we observe the output from LSB to MSB, the degree of the polynomial increases by 1 with each step, as expressed below:*

$$\deg(f_i) = \begin{cases} 2 & \text{for } i = 1, 2, \\ 4 & \text{for } i = 3, \\ \deg(f_{i-1}) + 1 & \text{for } i \geq 4. \end{cases}$$

Proof. To facilitate a more thorough analysis of the algebraic structure of $A \boxtimes_w B$, the operation can be reinterpreted as an w -dimensional Boolean function that takes $2w$ input variables: w bits from A and w bits from B . As a result, the output of $A \boxtimes_w B$ can be expressed as an w -bit vector, denoted by $\{f_w, f_{w-1}, \dots, f_2, f_1\}$, where the inputs are represented as $A = \{a_w, a_{w-1}, \dots, a_2, a_1\}$ and $B = \{b_w, b_{w-1}, \dots, b_2, b_1\}$. Each output bit f_i represents a function of the input bits and can be mathematically expressed as:

$$f_i = \bigoplus_{j=1}^i a_j b_{i-j+1}, \quad \text{for } 1 \leq i \leq w.$$

The modular addition in f_i involves a simplification using binary XOR operations, and any resulting carry terms are represented separately. These simplified expressions for f_i include the carry terms c_i^j , which represent higher-order interactions between the input bits. Then f_i can be expressed as:

$$f_i = \bigoplus_{j=1}^i a_j b_{i-j+1} \oplus \bigoplus_{m=1}^{\sum_{l=1}^{i-2} l} c_{i-2}^m, \quad \text{for } 1 \leq i \leq w.$$

Here, the carry terms c_i^j represent contributions from multiple overlapping bit products, as defined below:

$$\begin{aligned} c_1^1 &= a_1 a_2 b_1 b_2, & c_2^1 &= a_1 a_2 b_2 b_3, \\ c_2^2 &= (a_1 b_3 \oplus a_2 b_2) a_3 b_1, & c_2^3 &= (a_1 b_3 \oplus a_2 b_2 \oplus a_3 b_1) c_1^1. \end{aligned}$$

Substituting the carry terms back into the expressions for f_i , we obtain the following detailed expression for the first few output bits:

$$\begin{aligned} f_1 &= a_1 b_1, \\ f_2 &= a_1 b_2 \oplus a_2 b_1, \\ f_3 &= a_1 b_3 \oplus a_2 b_2 \oplus a_3 b_1 \oplus a_1 a_2 b_1 b_2, \\ f_4 &= a_1 b_4 \oplus a_2 b_3 \oplus a_3 b_2 \oplus a_4 b_1 \oplus a_1 a_2 b_2 b_3 \oplus a_1 a_3 b_1 b_3 \\ &\quad \oplus a_2 a_3 b_1 b_2 \oplus a_1 a_2 b_1 b_2 b_3 \oplus a_1 a_2 b_1 b_2 \oplus a_1 a_2 a_3 b_1 b_2, \\ &\vdots \end{aligned}$$

As we move from the least significant bit (LSB) to the most significant bit (MSB), the degree of the polynomial increases by 1, expressed as:

$$\deg(f_{i+1}) = \deg(f_i) + 1, \quad \forall 3 \leq i \leq w.$$

For f_1, f_2, f_3 , the degrees are $\deg(f_1) = 2$, $\deg(f_2) = 2$, and $\deg(f_3) = 4$, with two new variables introduced at each step, but never in the same monomial. $\square$

Effect of RFnc. In the MSX cipher, the inputs to the modulo addition operation are the data X and the key K . The first two modulo additions in the MSX cipher operate on the least significant $w/2$ bits of X , with the remaining bits set to zero. Assuming all k_i 's to be some constants, the following results hold:

Lemma 2. *Let the modulo addition take X and K as inputs, where $w/2$ MSBs of X are zero, such that $X = \{0, \dots, 0, x_{w/2}, \dots, x_1\}$. The degree increment is caused by the carry bits (c_i). For $1 \leq i \leq w/2$, c_i includes all possible combinations of variables, as c_{i-1} is multiplied by the current x_i 's. For c_i where $i > w/2$, the degree stabilizes, and the output reflects the contribution of the constants k_i , resulting in distinct coefficients for each term.*

Proof. In the MSX cipher, the first two modulo additions operate on the least significant $w/2$ elements of X , with the remaining elements of X being zeros. In this case, the degree condition $\deg(f_{i+1}) = \deg(f_i) + 1$ holds for $2 \leq i \leq w/2 + 1$, and $\deg(f_{w/2+1}) = w/2$. Next, consider the carry terms c_i , which are defined as

$$c_i = x_i k_i \oplus c_{i-1} x_i \oplus c_{i-1} k_i, \quad \forall 2 \leq i \leq w.$$

These carry terms represent the intermediate results of modulo addition at each stage. Their algebraic degree increases as the computation progresses, reflecting the growing interactions between X and K . As we move from LSB to MSB, the degree increases by one, i.e., $\deg(c_{i+1}) = \deg(c_i) + 1$ for $1 \leq i \leq w$. Treating all k_i 's as constants, this relation holds with $\deg(c_2) = 1$. In the MSX cipher, the first two modulo additions process the least significant $w/2$ elements of X , with the remaining elements set to zero.

Furthermore, as we move from LSB to MSB, each c_i incorporates all possible terms involving the variables x_{ij} such that $1 \leq j \leq i$. Let the set of constants be defined as

$$\mathcal{K} = \left\{ f(S) : S \subseteq \{1, 2, \dots, w\}, S \neq \emptyset \right\}, \text{ where } f(S) = \prod_{i \in S} (1 + k_i) - 1.$$

Expanding the product $\prod_{i \in S} (1 + k_i)$ yields the empty product 1 plus the sum of all nonempty products of the k_i 's. Hence, $f(S)$ compactly represents all possible nonempty multiplicative combinations of $\{k_i\}$.

After substituting the k_i 's with elements from $\mathcal{K}$, the equations for the c_i 's contain monomials with coefficients drawn from $\mathcal{K}$. For example,

$$c_3 = x_1 \mathcal{K} \oplus x_2 \mathcal{K} \oplus x_3 \mathcal{K} \oplus x_1 x_2 \mathcal{K} \oplus x_2 x_3 \mathcal{K} \oplus x_1 x_3 \mathcal{K} \oplus x_1 x_2 x_3 \mathcal{K}.$$

Note that each c_i generates all possible combinations of variables, as c_{i-1} is multiplied with the current x_i at every step for $1 \leq i \leq w/2$. For the remaining c_i 's, a similar pattern is observed, but with different coefficients arising from the constants k_i 's. $\square$

After observing the behaviour of the carry terms and their contribution to the algebraic structure, we now state a theorem that captures the resulting degree progression of the output bits.

Theorem 1. *Let the modulo addition take X and K , where $X = \{0, \dots, 0, x_{w/2}, \dots, x_1\}$, and assume all k_i 's are constants. Then, the degree progression of the output bits f_i is given by:*

$$\deg(f_i) = \begin{cases} 1 & \text{for } i = 1, 2, \\ \deg(f_{i-1}) + 1 & \text{for } 3 \leq i \leq w/2 + 1, \\ w/2 & \text{for } w/2 + 1 \leq i \leq w. \end{cases}$$

Furthermore, each f_i contains all possible monomials, including those contributing to the maximum-degree monomial.

This theorem can be proved using the combined results of Lemma 2 and Observation 1. When the output of modulo addition, under the above conditions, is used as input to modulo multiplication, the following result holds.

Theorem 2. *Consider the modulo multiplication where $H = \{h_w, \dots, h_1\}$ represents the output bits. Let Y and Z be the inputs, with the degree progression satisfying $\deg(y_{i+1}) = \deg(y_i) + 1$ for $1 \leq i \leq w/2$, and $\deg(y_j) = w/2$ for $w/2 < j \leq w$, with all possible monomials included, including those that form the maximum-degree monomial. The same conditions hold for Z . The degree progression of the output bits f_i is given by:*

$$\deg(h_i) = \begin{cases} > w & \text{for } i = w/2 + 1, \\ w & \text{for } i > w/2 + 1. \end{cases}$$

This theorem can be proved using the combined results of Lemma 1 and Theorem 1. After the multiplication step, the output bits h_i go through modulo addition, and only the 16 MSBs are kept. According to Theorem 2, except for two bits, all remaining bits reach the highest possible degree and include all types of monomials. Since the design uses XOR with rotated versions of the input (like $x + \text{ROL}(x, r)$), each round produces polynomials with maximum complexity. Therefore, the round function RFnc increases the algebraic degree of the input by 32 in every round.

Division Property by Algebraic Analysis. The division property, introduced by Todo [Tod15], serves as a generalization of integral distinguishers [DKR97, KW02] and higher-order differential distinguishers [Lai94, Knu94], and has been used to propose new generic distinguishers for both SPN and Feistel constructions. In the context of the MSX cipher, algebraic analysis of the propagation of degree across rounds provides important insights into the cipher's structure. The division property relies on this algebraic progression and helps in identifying distinguishers. After each round of MSX-64 and MSX-128, the algebraic degree increases by a maximum of 32 because of the RFnc. Consequently, MSX-64 reaches its maximum degree after 4 rounds, while MSX-128 achieves its maximum degree after 6 rounds. The propagation of the degree across rounds is detailed in the supplementary material. At these respective points, each bit in the state is expressed as a polynomial containing all possible monomials of the maximum degree, with coefficients determined by the encryption keys. Therefore, the division property necessitates the maximum number of variables after 4 rounds for MSX-64 and after 6 rounds for MSX. For earlier rounds, the number of variables required is reduced and is summarized in Table 4.

Table 4: Number of variables required for the division property at different rounds.

Rounds	1	2	3	4	5	6
MSX-64	1	1	32	†		
MSX-128	1	1	1	32	32	†

† Highest degree attained.

5.4 Generic Attacks

The security of MSX is analyzed against various generic attacks on Feistel ciphers. Specifically, MSX-64 and MSX-128 represent instantiations of balanced Feistel and Type-2 generalized Feistel structures (GFS), respectively. The impossible differential attacks in [KHL10] on both balanced Feistel and Type-2 Feistel and the differential attacks on balanced Feistel in [CCP24, TP09, Knu02] are not applicable on all variants of MSX, as these attacks consider bijective round function. The differential attack in [Pat08] are applicable upto 5 rounds of MSX-64. By considering the round functions as 32-bit secret S-boxes (non-bijective), yoyo attacks can be mounted on 7-round MSX-64 to recover these S-boxes [BLP15] with a data complexity of 2^{64} and a time complexity significantly larger ($2^{32 \times 2^{32} + 2 \times 32} = 2^{2^{37}}$). Yoyo attacks on Type-2 GFS are applicable to 7-round MSX-128, with the corresponding data and time complexities being 2^{98} and 2^{136} , respectively. SAT-based approach to recover the secret S-boxes [BP15] are not applicable on MSX-64 as the attack is feasible only when the word length is less than 8.

5.5 Impossible Differential

In an impossible differential attack [BBS99], an attacker attempts to identify an input difference that will never propagate to a specific output difference. To demonstrate the resistance of MSX against impossible differential attacks, we follow the approach outlined in [BBI⁺15]. The REnc function achieves full diffusion in just one execution. Due to the Feistel structure of MSX, there is no probability-one truncated differential trail for three rounds of MSX-64 and five rounds of MSX-128. Using the miss-in-the-middle approach, the maximum number of rounds for which an impossible differential characteristic exists is estimated to be 7 and 11 for MSX-64 and MSX-128, respectively. Hence, we believe full rounds of MSX-64 and MSX-128 provides enough resistance against impossible differential attack.

5.6 Rotational Analysis

Rotational cryptanalysis, introduced by Khovratovich and Nikolic [KN10], is a cryptanalytic technique in which the adversary requests the encryption of a pair of plaintexts, where one plaintext is derived from the other through a cyclic rotation. Consider $x, y \in \mathbb{F}_2^w$. In this analysis, we are primarily interested in estimating the probability of the event $\text{REnc}(\text{ROR}(x, r)) = \text{ROR}(\text{REnc}(x), r)$. From [Dau05], we know that $\Pr[\text{ROR}((x \boxplus_w y), r) = \text{ROR}(x, r) \boxplus_w (\text{ROR}(y, r))] = \frac{1}{4}(1 + 2^{r-w} + 2^{-r} + 2^{-w})$. However, the $\boxplus_w$ operation resists in furthering the rotational invariance properties. More specifically, we found no relation between $\text{ROR}((x \boxtimes_w y), r)$ and $\text{ROR}(x, r) \boxtimes_w \text{ROR}(y, r)$. Consider, $z = (x \boxtimes_w y)$, $z' = \text{ROR}(z, r)$, $x' = \text{ROR}(x, r)$, $y' = \text{ROR}(y, r)$ and $z'' = (\text{ROR}(x, r)) \boxtimes_w (\text{ROR}(y, r))$. Then, $z'_i = z_{i-r} = \sum_{j=0}^{i-r} x_j y_{i-r-j} \oplus c_{i-r-1}$. Similarly, $z''_i = \sum_{j=0}^i x'_j y'_{i-j} \oplus c'_{i-1} = \sum_{j=0}^i x_{j-r} y_{i-r-j} \oplus c_{i-r-1}$. It is evident from the above two expressions that $z'_i = z''_i$ only when $r = 0$. Thus, $\boxtimes_w$ operation destroys any rotational invariance property. Note that, in MSX, the rotation operation is realized by the operations msb_{16} and lsb_{16} .

5.7 Boomerang and Differential-Linear Attacks

The boomerang attack [Wag99], introduced by Wagner, extends differential attacks by splitting a cipher E into two parts, E_1 and E_0 , allowing the attack to penetrate additional rounds. Initially, independent differential trails for E_0 and E_1 were combined to construct a boomerang trail. Later studies revealed dependencies between these trails [Mur11], leading to frameworks with an intermediate layer E_m [BK09, DKS10, DKS14]. The *boomerang connectivity table* (BCT) was introduced to capture these dependencies for one-round E_m [CHP⁺18], and subsequent works analyzed multi-round E_m [WP19, DDV20, DEFN22, HNE22]. For MSX-64, with two rounds each for E_0 and E_1 , a 4-round boomerang trail has a maximum probability of 2^{-144} . Even with a 2-round deterministic E_m [WWS23], a 6-round boomerang distinguisher has the same maximum probability. Similarly, for MSX-128, the maximum probability for a 10-round boomerang distinguisher is 2^{-152} .

Differential-linear cryptanalysis [LH94] identifies an input difference-output mask pair $(\Delta_{in}, \Gamma_{out})$ and analyzes the bias ϵ in the approximation of ciphertext differences for plaintext pairs with input difference Δ_{in} . The cipher E is divided into E_0 and E_1 , where E_0 follows a differential trail, and E_1 involves a linear approximation. Dependencies between trails in middle rounds (E_m), as discussed in [BDK02], are captured by the Differential-Linear Connectivity Table (DLCT) [BDKW19], with similar studies for ARX ciphers in [BGG⁺23, HDE24]. For MSX-64 and MSX-128, considering a 2-round E_m , the maximum probabilities for 6-round and 10-round differential-linear trails are 2^{-144} and 2^{-152} , respectively. Therefore, the full-round versions of MSX-64 and MSX-128 are resistant to both boomerang and differential-linear attacks.

5.8 Weak keys

The possibility of weak key attacks in MSX is significantly reduced due to the introduction of constants in both the round function and the key scheduling operation. Consider a scenario in which the round constants and key constants are omitted- under such conditions, specific distinguishers can be constructed, for instance, when the key is set to zero. However, due to the inclusion of these constants, such attacks do not propagate beyond a single round. A similar effect of round constants is observed in the context of algebraic analysis: if the constants are omitted from the round function and key schedule, it takes more rounds to achieve the maximum algebraic degree for certain keys. Moreover, it appears that RFnc exhibits specific properties due to the modular addition operation. In particular, the relation $\text{RFnc}(x, K_1, K_2, K_3, K_4, K_5, K_6) = \text{RFnc}(x \oplus a, K_1 \oplus b, K_2, K_3, K_4 \oplus b, K_5, K_6)$ holds with probability $\frac{1}{2}$, where $a = 0x001F$ and $b = 0x000F$. However, such a property can be exploited to attack at most one round in the weak-key setting.

6 Implementation Results

We conducted a benchmark on the following 32-bit AVR/ARM microcontrollers.

- 32-bit AVR: Atmel AT32UC3A0128 (App./Boot Memory 128Kbytes, Data Memory 32Kbytes)²
- 32-bit ARM Cortex-M3 (STMicro NUCLEO F103RB)³
- 32-bit ARM Cortex-M4 (STMicro NUCLEO F411RE)⁴

²<https://www.microchip.com/en-us/product/at32uc3a0128>

³<https://www.st.com/en/evaluation-tools/nucleo-f103rb.html>

⁴<https://www.st.com/en/evaluation-tools/nucleo-f411re.html>

We implemented and benchmarked all members of MSX under pre-computed round key settings. The pre-computed round key setting is relevant for most of the use cases where the key will be used for a certain period. Since our target platforms are capable of caching round keys, we choose this as our benchmark scenario. Following [ANP20, AP21], we also benchmarked the key schedule itself, which enables us to estimate performance under the on-the-fly key setting. For comparison, we also benchmarked Speck-x-y having x-bit block and y-bit key, for $(x, y) \in \{(64, 128), (128, 128), (128, 256)\}$. We entirely use C to implement MSX and Speck without (inline) assembly. The C source of Speck is taken from the official repository [spe21] that is used by Supercop [BL18]. For Speck, we applied a minor edit to the input format (from word to byte) to align with our implementation of MSX. The top-level interface follows the one used by FELICS benchmark framework [DCK⁺15, DCK⁺19]. For AVR, we used a cycle-accurate official simulator tool (Microchip Studio), following previous studies, *e.g.* [IMG15, BSS⁺14]. We use `avr-gcc` (version 4.4.7) for compilation. For ARM, we use STM32CubeIDE⁵ for development environment and use `gcc` (`arm-none-eabi-gcc`, version 10.3.1) for compilation. For all cases, RFnc is compiled into a sequence of constant-time instructions. For example, on Cortex-M3/M4, MLA instruction is used, which merges an addition and a half multiplication and always runs in 2 cycles. The 32-bit AVR has `mul` instruction for half multiplication running at 1 cycle.⁶

For speed-first implementation, we tried several options for the depth of loop unrolling and picked the fastest one when compiled with `-O3` option. For the most of cases, a fully unrolled code was chosen. For size-first implementation, the code is loop-based and compiled with `-O2` option. Table 5 shows the results of 64-bit block ciphers (MSX-64 and Speck-64-128, and Table 6 show the results of 128-bit block ciphers (MSX-128-128, MSX-128-256, Speck-128-128, and Speck-128-256). In these tables, **Code/RAM** is measured in bytes and **Time** denotes cycles. For MSX and Speck, our implementation stores their round keys at RAM. The **RAM** column shows $x + y$ when the amount of used stacks is x bytes and that of the stored round keys is y bytes.

Further investigation on performance figures. We provide a detailed look on the performance figures of Speck and MSX in Tables 5 and 6. MSX incurs more cycles than Speck, and in particular for “speed-first” implementation on ARM M3 and M4, we observed that MSX-64 incurs approximately 2.7 more cycles compared to Speck-64-128. We estimate the required cycles from the the assembly codes with inclusion of push/pop operations. The result is shown in Table. 9. On the estimated basis, MSX-64 has 312 instructions, which is about 2.42 times more than Speck-64-128. Among these, 122 are two-cycle instructions, such as load/store and multiply-and-add operations, exceeding Speck-64-128 by 85 instructions. The increased number of load instructions is attributed to the larger size of the total round keys in MSX-64. Estimated cycle ratio based on these figures is 2.71, which is close to the actual benchmark result. We observe that the measured cycle count is slightly smaller than estimated; this might be the effect of pipelined memory read. In the case of 128-bit block ciphers, MSX-128-128 exhibits a performance loss of approximately 1.5 times compared with Speck-128-128. The number of instructions for MSX-128-128 is 774. Since the operation size of RFnc is the same as that of MSX-64, the cycle count is simply proportional to the number of RFnc executions. In each round, two instances of RFnc operate in parallel, and the linear layer is processed through pipelining, which leads to a slight improvement in execution efficiency (unlike the case of MSX-64 where linear layer of RFnc must wait for the non-linear layer). Speck-128-128, on the other hand, has 560 instructions, which is four times that of Speck-64-128. Since Speck-128-128 operates on 64-bit units, basic operations such as addition and exclusive OR require two

⁵<https://www.st.com/en/development-tools/stm32cubeide.html>

⁶<https://ww1.microchip.com/downloads/en/DeviceDoc/doc32000.pdf>

Table 5: Implementation result of 64-bit block ciphers

	AVR-32			Cortex-M3			Cortex-M4		
	Code	RAM	Time	Code	RAM	Time	Code	RAM	Time
MSX-64 (size first)									
Key schedule	456	84+336	619	420	112+336	725	392	112+336	626
Encryption	236	52+336	588	156	28+336	510	164	28+336	491
Decryption	244	52+336	592	160	28+336	510	176	28+336	521
MSX-64 (speed first)									
Key schedule	874	40+336	285	492	36+336	218	504	36+336	210
Encryption	766	68+336	520	516	32+336	440	516	32+336	440
Decryption	790	68+336	524	516	32+336	446	516	32+336	446
Speck-64-128 (size first)									
Key schedule	156	12+108	492	76	16+108	203	76	16+108	208
Encryption	126	8+108	403	76	4+108	273	76	4+108	278
Decryption	126	8+108	430	80	4+108	302	72	4+108	331
Speck-64-128 (speed first)									
Key schedule	398	36+108	317	380	4+108	134	384	4+108	137
Encryption	346	36+108	300	380	4+108	162	376	4+108	167
Decryption	352	36+108	301	492	4+108	192	484	4+108	195

Table 6: Implementation result of 128-bit block 128-bit key ciphers

	AVR-32			Cortex-M3			Cortex-M4		
	Code	RAM	Time	Code	RAM	Time	Code	RAM	Time
MSX-128-128 (size first)									
Key schedule	742	164+864	1387	804	208+864	1993	708	216+864	1815
Encryption	486	104+864	1383	276	48+864	1299	276	48+864	1198
Decryption	512	104+864	1383	292	48+864	1301	288	48+864	1199
MSX-128-128 (speed first)									
Key schedule	2270	100+864	744	1592	112+864	714	1492	104+864	651
Encryption	1832	100+864	1166	1240	56+864	1092	1260	56+864	1042
Decryption	1832	100+864	1166	1308	56+864	1079	1324	56+864	1030
Speck-128-128 (size first)									
Key schedule	364	124+256	1026	108	16+256	1133	100	16+256	745
Encryption	528	136+256	1021	164	20+256	1007	168	20+256	731
Decryption	474	104+256	992	168	16+256	1081	164	20+256	760
Speck-128-128 (speed first)									
Key schedule	2168	80+256	753	1564	20+256	687	1540	16+256	523
Encryption	2532	396+256	916	512	24+256	799	512	24+256	653
Decryption	2544	396+256	932	556	28+256	870	556	28+256	651

instructions each. Moreover, 64-bit rotation shifts require four instructions. As each round function of **Speck** includes two rotation shifts, the increase in instructions related to this operation becomes a major factor in the performance degradation of **Speck-128-128**. Consequently, the difference in instruction count between **MSX-128-128** and **Speck-128-128** becomes relatively small.

Considering that **Speck** is one of the top performers on microcontrollers, these results suggest a positive characteristic for **MSX** as an alternative to existing designs for microcontroller-oriented designs. We also benchmarked AES implemented by the state-of-the-art fixslicing [AP21] technique using the codes from [aes20]. These codes are assembly and highly optimized to M3 and M4. Table 8 shows the results, indicating the advantage of **MSX**. We note that the figures in Table 8 exhibits some differences from those reported in the original paper [AP21] that should reflect the specific characteristics of our platforms.

Table 7: Implementation result of 128-bit block 256-bit key ciphers

	AVR-32			Cortex-M3			Cortex-M4		
	Code	RAM	Time	Code	RAM	Time	Code	RAM	Time
MSX-128-256 (size first)									
Key schedule	930	164+256	1400	840	216+256	2005	728	208+256	1780
Encryption	486	104+256	1385	276	48+864	1299	276	48+864	1196
Decryption	512	104+864	1381	292	48+864	1301	288	48+864	1197
MSX-128-256 (speed first)									
Key schedule	2450	96+256	803	1664	112+256	715	1620	112+256	682
Encryption	1832	100+864	1168	1240	56+864	1092	1260	56+864	1042
Decryption	1882	100+864	1164	1308	56+864	1079	1324	56+864	1030
Speck-128-256 (size first)									
Key schedule	758	120+272	1511	260	36+272	777	256	36+272	728
Encryption	492	136+272	1044	164	20+272	814	168	24+272	773
Decryption	452	108+272	1028	168	16+272	812	164	20+272	804
Speck-128-256 (speed first)									
Key schedule	2616	164+272	925	1748	36+272	597	1792	36+272	589
Encryption	2666	412+272	957	576	24+272	708	576	24+272	701
Decryption	2690	412+272	983	536	20+272	692	536	20+272	681

Table 8: Implementation result of AES

	Parallel blocks	Time (cycles/block)		ROM (bytes)		RAM (bytes)	
		M3	M4	Code	Data	In/Out	Stack
AES-128 Key schedule (fixsliced)							
semi-fixsliced	2	3769	3741	1608	0	384	112
fully-fixsliced	2	3954	3925	1752	0	384	112
AES-128 Key schedule (LUT based)							
semi-fixsliced	1	1498	1484	756	256	368	212
fully-fixsliced	1	1520	1535	952	256	368	212
barrel-shiftrows	1	2896	2885	684	256	1424	216
AES-256 Key schedule (fixsliced)							
semi-fixsliced	2	4939	4946	2156	0	544	112
fully-fixsliced	2	5212	5248	2304	0	544	112
AES-256 Key schedule (LUT based)							
semi-fixsliced	1	2069	2069	1086	256	512	264
fully-fixsliced	1	2102	2102	1226	256	512	264
barrel-shiftrows	1	3985	3987	804	256	1952	280
AES-128 Encryption							
semi-fixsliced	2	1422	1422	1856	256	32(+352)	116
fully-fixsliced	2	1359	1360	2396	256	32(+352)	116
barrel-shiftrows	8	1516	1513	2532	256	128(+1408)	244
AES-256 Encryption							
semi-fixsliced	2	1935	1936	1896	256	32(+480)	116
fully-fixsliced	2	1864	1864	2428	256	32(+480)	116
barrel-shiftrows	8	2054	2051	2532	256	128(+1920)	244

7 Concluding Remarks

The proposed MSX shows good performance by utilizing a strong universal hash function based on integer multiplication and ensures strong immunity against differential and linear attacks by design. Together with our in-depth analysis against other attacks, we think the

Table 9: Cycle count analysis by assembly codes. The upper half expresses the case of 64-bit block ciphers. The lower half expresses the case of 128-bit block, 128-bit key ciphers. The platform is ARM M3 and implementation scenario is “speed-first” for both.

	Cycles		Number of Est. Instructions*
	Measured	Estimated	
MSX-64 (<i>a</i>)	440	450	312 (122)
Speck-64-128 (<i>b</i>)	162	166	129 (37)
Difference (<i>a</i> − <i>b</i>)	278	284	183 (85)
Ratio (<i>a</i> / <i>b</i>)	2.72	2.71	2.42 (3.30)
MSX-128-128 (<i>a</i>)	1092	1022	774 (230)
Speck-128-128 (<i>b</i>)	799	654	560 (84)
Difference (<i>a</i> − <i>b</i>)	293	368	214 (85)
Ratio (<i>a</i> / <i>b</i>)	1.37	1.56	1.38 (2.74)

* $x(y)$ denotes $x - y$ single cycle instructions and y two-cycle instructions.

current specifications are sufficiently strong.

We, however, emphasize that implementation must be done with care about timing side channels, particularly about 32-bit half integer multiplication. While the platforms we tested perform constant-time multiplication, this may not hold universally and constant-time operation should be checked before real use, as in the case of other ciphers. Since we strictly keep the round function to be strongly uniform, it involves a large number of subkeys. This incurs non-negligible performance loss on microcontrollers for many load/store instructions. Whether we could reduce the number of subkeys, maintaining a strong guarantee against differential/linear attacks, is an interesting open question. To summarize, we suggest the following list to clarify the advantages and disadvantages of our design. Our advantages:

- Theoretical resistance against differential and linear attacks by a simple and classic argument with a common assumption (uniform round keys)
- Fast execution utilizing the 32-bit half multiplication, which is built-in on many 32-bit microcontrollers
- Classical (generalized) Feistel structure, which enables to reuse many existing generic security analyses

Our disadvantages:

- Relatively high memory consumption due to the large memory of each round function, which can impact performance by lots of load/store instructions
- Limited suitability for platforms without a built-in 32-bit half multiplier
- Side-channel protection beyond constant time operation (e.g. masking against power analysis), which may be cumbersome due to multiplication

We elaborate the last item in the disadvantage list. On platforms where multiplication does not exhibit data-independent execution latency, the behavior of the multiplication instruction- such as its timing- may depend on the input values, which can introduce side-channel leakage. Mitigating this leakage generally requires applying countermeasures such as masking directly to the multiplication operation. Masked multiplication is inherently more complex, requiring additional randomness and extra intermediate computations to maintain security [KA24]. These factors increase the implementation cost and may reduce

performance, particularly on constrained platforms, making the practicality of protected implementations more platform-dependent.

Further exploration of our design approach to improve the overall merit would be an interesting direction. Extending our approach to other primitives such as cryptographic permutation is also an interesting future topic.

Acknowledgments

We sincerely thank the shepherd and anonymous reviewers for their helpful comments. We thank Hiroyasu Kubo and Maki Shigeri for the early-stage design considerations and experiments. This work was supported by JST CRONOS, Japan and JSPS KAKENHI, Grant Number JP24H00696.

References

- [AB12] Jean-Philippe Aumasson and Daniel J. Bernstein. SipHash: A fast short-input PRF. In Steven D. Galbraith and Mridul Nandi, editors, *INDOCRYPT 2012*, volume 7668 of *LNCS*, pages 489–508. Springer, Berlin, Heidelberg, December 2012. doi:10.1007/978-3-642-34931-7_28.
- [aes20] Fast constant-time AES implementations on 32-bit architectures, 2020. URL: <https://github.com/aadomn/aes>.
- [ALLW14] Farzaneh Abed, Eik List, Stefan Lucks, and Jakob Wenzel. Differential Cryptanalysis of Round-Reduced Simon and Speck. In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers*, volume 8540 of *Lecture Notes in Computer Science*, pages 525–545. Springer, 2014. doi:10.1007/978-3-662-46706-0_27.
- [AMPH14] Jean-Philippe Aumasson, Willi Meier, Raphael C.-W. Phan, and Luca Henzen. *The Hash Function BLAKE*. Information Security and Cryptography. Springer, 2014.
- [ANP20] Alexandre Adomnicai, Zakaria Najm, and Thomas Peyrin. Fixslicing: A new gift representation. *IACR TCHES*, 2020(3):402–427, 2020. URL: <https://tches.iacr.org/index.php/TCHES/article/view/8595>, doi:10.13154/tches.v2020.i3.402-427.
- [AO97] Kazumaro Aoki and Kazuo Ohta. Strict Evaluation of the Maximum Average of Differential Probability and the Maximum Average of Linear Probability. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 80(1):2–8, 1997.
- [AP21] Alexandre Adomnicai and Thomas Peyrin. Fixslicing AES-like ciphers. *IACR TCHES*, 2021(1):402–425, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/8739>, doi:10.46586/tches.v2021.i1.402-425.
- [BBdS⁺19] Christof Beierle, Alex Biryukov, Luan Cardoso dos Santos, Johann Großschädl, Léo Perrin, Aleksei Udovenko, Vesselin Velichkov, Qingju Wang, Amir Moradi, and Aein Rezaei Shahmirzadi. SPARKLE (SCHWAEMM and ESCH). A submission to NIST Lightweight Cryptography, 2019.
- [BBdS⁺20] Christof Beierle, Alex Biryukov, Luan Cardoso dos Santos, Johann Großschädl, Léo Perrin, Aleksei Udovenko, Vesselin Velichkov, and Qingju Wang. Alzette: A

- 64-bit ARX-box - (feat. CRAX and TRAX). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 419–448. Springer, Cham, August 2020. doi:[10.1007/978-3-030-56877-1_15](https://doi.org/10.1007/978-3-030-56877-1_15).
- [BBI⁺15] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A Block Cipher for Low Energy. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part II*, volume 9453 of *Lecture Notes in Computer Science*, pages 411–436. Springer, 2015. doi:[10.1007/978-3-662-48800-3_17](https://doi.org/10.1007/978-3-662-48800-3_17).
- [BBI⁺20] Subhadeep Banik, Zhenzhen Bao, Takanori Isobe, Hiroyasu Kubo, Fukang Liu, Kazuhiko Minematsu, Kosei Sakamoto, Nao Shibata, and Maki Shigeri. WARP : Revisiting GFN for lightweight 128-bit block cipher. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 535–564. Springer, Cham, October 2020. doi:[10.1007/978-3-030-81652-0_21](https://doi.org/10.1007/978-3-030-81652-0_21).
- [BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 12–23. Springer, Berlin, Heidelberg, May 1999. doi:[10.1007/3-540-48910-X_2](https://doi.org/10.1007/3-540-48910-X_2).
- [BCJW02] Nikita Borisov, Monica Chew, Robert Johnson, and David A. Wagner. Multiplicative Differentials. In Joan Daemen and Vincent Rijmen, editors, *Fast Software Encryption, 9th International Workshop, FSE 2002, Leuven, Belgium, February 4-6, 2002, Revised Papers*, volume 2365 of *Lecture Notes in Computer Science*, pages 17–33. Springer, 2002. doi:[10.1007/3-540-45661-9_2](https://doi.org/10.1007/3-540-45661-9_2).
- [BDK02] Eli Biham, Orr Dunkelman, and Nathan Keller. Enhancing Differential-Linear Cryptanalysis. In Yuliang Zheng, editor, *Advances in Cryptology - ASIACRYPT 2002, 8th International Conference on the Theory and Application of Cryptology and Information Security, Queenstown, New Zealand, December 1-5, 2002, Proceedings*, volume 2501 of *Lecture Notes in Computer Science*, pages 254–266. Springer, 2002. doi:[10.1007/3-540-36178-2_16](https://doi.org/10.1007/3-540-36178-2_16).
- [BDKW19] Achiya Bar-On, Orr Dunkelman, Nathan Keller, and Ariel Weizman. DLCT: A new tool for differential-linear cryptanalysis. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I*, volume 11476 of *Lecture Notes in Computer Science*, pages 313–342. Springer, 2019. doi:[10.1007/978-3-030-17653-2_11](https://doi.org/10.1007/978-3-030-17653-2_11).
- [Ber05] Daniel J. Bernstein. The poly1305-AES message-authentication code. In Henri Gilbert and Helena Handschuh, editors, *FSE 2005*, volume 3557 of *LNCS*, pages 32–49. Springer, Berlin, Heidelberg, February 2005. doi:[10.1007/11502760_3](https://doi.org/10.1007/11502760_3).
- [Ber08] Daniel J. Bernstein. ChaCha, a variant of Salsa20. <https://cr.yp.to/cha/cha/chacha-20080128.pdf>, January 2008.
- [BF07] Thomas Baignères and Matthieu Finiasz. Dial C for cipher. In Eli Biham and Amr M. Youssef, editors, *SAC 2006*, volume 4356 of *LNCS*, pages 76–95.

- Springer, Berlin, Heidelberg, August 2007. doi:[10.1007/978-3-540-74462-7_7](https://doi.org/10.1007/978-3-540-74462-7_7).
- [BFL⁺21] Olivier Bronchain, Sebastian Faust, Virginie Lallemand, Gregor Leander, Léo Perrin, and François-Xavier Standaert. MOE: Multiplication operated encryption with trojan resilience. *IACR Trans. Symm. Cryptol.*, 2021(1):78–129, 2021. doi:[10.46586/tosc.v2021.i1.78-129](https://doi.org/10.46586/tosc.v2021.i1.78-129).
- [BGG⁺23] Emanuele Bellini, David G rault, Juan Grados, Rusydi H. Makarim, and Thomas Peyrin. Fully Automated Differential-Linear Attacks Against ARX Ciphers. In Mike Rosulek, editor, *Topics in Cryptology - CT-RSA 2023 - Cryptographers' Track at the RSA Conference 2023, San Francisco, CA, USA, April 24-27, 2023, Proceedings*, volume 13871 of *Lecture Notes in Computer Science*, pages 252–276. Springer, 2023. doi:[10.1007/978-3-031-30872-7_10](https://doi.org/10.1007/978-3-031-30872-7_10).
- [BHK⁺99] John Black, Shai Halevi, Hugo Krawczyk, Ted Krovetz, and Phillip Rogaway. UMAC: Fast and secure message authentication. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 216–233. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-48405-1_14](https://doi.org/10.1007/3-540-48405-1_14).
- [BK09] Alex Biryukov and Dmitry Khovratovich. Related-Key Cryptanalysis of the Full AES-192 and AES-256. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2009. doi:[10.1007/978-3-642-10366-7_1](https://doi.org/10.1007/978-3-642-10366-7_1).
- [BL18] Daniel J. Bernstein and Tanja Lange. Supercop – eBACS: ECRYPT Benchmarking of Cryptographic Systems, 2018. URL: <https://bench.cr.yp.to>.
- [BLP15] Alex Biryukov, Ga tan Leurent, and L o Perrin. Cryptanalysis of Feistel Networks with Secret Round Functions. In Orr Dunkelman and Liam Keliher, editors, *Selected Areas in Cryptography - SAC 2015 - 22nd International Conference, Sackville, NB, Canada, August 12-14, 2015, Revised Selected Papers*, volume 9566 of *Lecture Notes in Computer Science*, pages 102–121. Springer, 2015. doi:[10.1007/978-3-319-31301-6_6](https://doi.org/10.1007/978-3-319-31301-6_6).
- [BP15] Alex Biryukov and L o Perrin. On Reverse-Engineering S-Boxes with Hidden Design Criteria or Structure. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 116–140. Springer, 2015. doi:[10.1007/978-3-662-47989-6_6](https://doi.org/10.1007/978-3-662-47989-6_6).
- [BPP⁺17] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In Wieland Fischer and Naofumi Homma, editors, *CHES 2017*, volume 10529 of *LNCS*, pages 321–345. Springer, Cham, September 2017. doi:[10.1007/978-3-319-66787-4_16](https://doi.org/10.1007/978-3-319-66787-4_16).
- [BRV14] Alex Biryukov, Arnab Roy, and Vesselin Velichkov. Differential Analysis of Block Ciphers SIMON and SPECK. In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers*, volume 8540 of *Lecture Notes in Computer Science*, pages 546–570. Springer, 2014. doi:[10.1007/978-3-662-46706-0_28](https://doi.org/10.1007/978-3-662-46706-0_28).

- [BS15] Adnan Baysal and Sühap Sahin. Roadrunner: A small and fast bitslice block cipher for low cost 8-bit processors. In Tim Güneysu, Gregor Leander, and Amir Moradi, editors, *Lightweight Cryptography for Security and Privacy - 4th International Workshop, LightSec 2015, Bochum, Germany, September 10-11, 2015, Revised Selected Papers*, volume 9542 of *Lecture Notes in Computer Science*, pages 58–76. Springer, 2015. doi:[10.1007/978-3-319-29078-2_4](https://doi.org/10.1007/978-3-319-29078-2_4).
- [BSS⁺14] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The simon and speck block ciphers on AVR 8-bit microcontrollers. In Thomas Eisenbarth and Erdiñç Öztürk, editors, *Lightweight Cryptography for Security and Privacy - Third International Workshop, LightSec 2014, Istanbul, Turkey, September 1-2, 2014, Revised Selected Papers*, volume 8898 of *Lecture Notes in Computer Science*, pages 3–20. Springer, 2014. doi:[10.1007/978-3-319-16363-5_1](https://doi.org/10.1007/978-3-319-16363-5_1).
- [BSS⁺15] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The SIMON and SPECK lightweight block ciphers. In *Proceedings of the 52nd Annual Design Automation Conference, San Francisco, CA, USA, June 7-11, 2015*, pages 175:1–175:6. ACM, 2015. doi:[10.1145/2744769.2747946](https://doi.org/10.1145/2744769.2747946).
- [BV14] Alex Biryukov and Vesselin Velichkov. Automatic Search for Differential Trails in ARX Ciphers. In Josh Benaloh, editor, *Topics in Cryptology - CT-RSA 2014 - The Cryptographer’s Track at the RSA Conference 2014, San Francisco, CA, USA, February 25-28, 2014. Proceedings*, volume 8366 of *Lecture Notes in Computer Science*, pages 227–250. Springer, 2014. doi:[10.1007/978-3-319-04852-9_12](https://doi.org/10.1007/978-3-319-04852-9_12).
- [CCP24] Maya Chartouny, Benoît Cogliati, and Jacques Patarin. Classical and quantum generic attacks on 6-round feistel schemes. In Maria Eichlseder and Sébastien Gambs, editors, *SAC 2024, Part II*, volume 15517 of *LNCS*, pages 85–111. Springer, Cham, August 2024. doi:[10.1007/978-3-031-82841-6_4](https://doi.org/10.1007/978-3-031-82841-6_4).
- [CGS17] Debrup Chakraborty, Sebati Ghosh, and Palash Sarkar. A fast single-key two-level universal hash function. *IACR Trans. Symm. Cryptol.*, 2017(1):106–128, 2017. doi:[10.13154/tosc.v2017.i1.106-128](https://doi.org/10.13154/tosc.v2017.i1.106-128).
- [CHP⁺18] Carlos Cid, Tao Huang, Thomas Peyrin, Yu Sasaki, and Ling Song. Boomerang Connectivity Table: A New Cryptanalysis Tool. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 683–714. Springer, 2018. doi:[10.1007/978-3-319-78375-8_22](https://doi.org/10.1007/978-3-319-78375-8_22).
- [CLLL00] Dong Hyeon Cheon, Sangjin Lee, Jong In Lim, and Sung Jae Lee. New block cipher DONUT using pairwise perfect decorrelation. In Bimal K. Roy and Eiji Okamoto, editors, *INDOCRYPT 2000*, volume 1977 of *LNCS*, pages 262–270. Springer, Berlin, Heidelberg, December 2000. doi:[10.1007/3-540-44495-5_23](https://doi.org/10.1007/3-540-44495-5_23).
- [Dau05] Magnus Daum. *Cryptanalysis of Hash Functions of the MD4-Family*. PhD thesis, Ruhr-Universität Bochum, May 2005.
- [DCK⁺15] Daniel Dinu, Yann Le Corre, Dmitry Khovratovich, Léo Perrin, Johann Großschädl, and Alex Biryukov. FELICS - Fair Evaluation of Lightweight

- Cryptographic Systems, 2015. URL: <https://www.cryptolux.org/index.php/FELICS>.
- [DCK⁺19] Daniel Dinu, Yann Le Corre, Dmitry Khovratovich, Léo Perrin, Johann Großschädl, and Alex Biryukov. Triathlon of lightweight block ciphers for the internet of things. *Journal of Cryptographic Engineering*, 9(3):283–302, September 2019. doi:10.1007/s13389-018-0193-x.
- [DDV20] Stéphanie Delaune, Patrick Derbez, and Mathieu Vavrille. Catching the Fastest Boomerangs Application to SKINNY. *IACR Trans. Symmetric Cryptol.*, 2020(4):104–129, 2020. doi:10.46586/TOSC.V2020.I4.104-129.
- [DEFN22] Patrick Derbez, Marie Euler, Pierre-Alain Fouque, and Phuong Hoa Nguyen. Revisiting Related-Key Boomerang Attacks on AES Using Computer-Aided Tool. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III*, volume 13793 of *Lecture Notes in Computer Science*, pages 68–88. Springer, 2022. doi:10.1007/978-3-031-22969-5_3.
- [DEMS19] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon. A submission to NIST Lightweight Cryptography, 2019.
- [dG15] Wouter de Groot. A performance study of X25519 on Cortex-M3 and M4. Master Thesis, 2015. URL: <https://pure.tue.nl/ws/portalfiles/porta1/47038543/800603-1.pdf>.
- [Die96] Martin Dietzfelbinger. Universal Hashing and k-Wise Independent Random Variables via Integer Arithmetic without Primes. In *STACS*, volume 1046 of *Lecture Notes in Computer Science*, pages 569–580. Springer, 1996.
- [Din14] Itai Dinur. Improved Differential Cryptanalysis of Round-Reduced Speck. In Antoine Joux and Amr M. Youssef, editors, *Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14-15, 2014, Revised Selected Papers*, volume 8781 of *Lecture Notes in Computer Science*, pages 147–164. Springer, 2014. doi:10.1007/978-3-319-13051-4_9.
- [DKR97] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The Block Cipher Square. In Eli Biham, editor, *Fast Software Encryption, 4th International Workshop, FSE '97, Haifa, Israel, January 20-22, 1997, Proceedings*, volume 1267 of *Lecture Notes in Computer Science*, pages 149–165. Springer, 1997. URL: <https://doi.org/10.1007/BFb0052343>, doi:10.1007/BFb0052343.
- [DKS10] Orr Dunkelman, Nathan Keller, and Adi Shamir. A Practical-Time Related-Key Attack on the KASUMI Cryptosystem Used in GSM and 3G Telephony. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 393–410. Springer, 2010. doi:10.1007/978-3-642-14623-7_21.
- [DKS14] Orr Dunkelman, Nathan Keller, and Adi Shamir. A Practical-Time Related-Key Attack on the KASUMI Cryptosystem Used in GSM and 3G Telephony. *J. Cryptol.*, 27(4):824–849, 2014. doi:10.1007/s00145-013-9154-9.
- [DPAR00] Joan Daemen, Michaël Peeters, Gilles Van Assche, and Vincent Rijmen. Nessie Proposal: NOEKEON, 2000. URL: <https://gro.noekeon.org/>.

- [DPU⁺16] Daniel Dinu, Léo Perrin, Aleksei Udovenko, Vesselin Velichkov, Johann Großschädl, and Alex Biryukov. Design strategies for ARX with provable bounds: Sparx and LAX. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 484–513. Springer, Berlin, Heidelberg, December 2016. doi:[10.1007/978-3-662-53887-6_18](https://doi.org/10.1007/978-3-662-53887-6_18).
- [FLS⁺10] Niels Ferguson, Stefan Lucks, Bruce Schneier, Doug Whiting, Mihir Bellare, Tadayoshi Kohno, Jon Callas, and Jesse Walker. The Skein Hash Function Family, 2010.
- [Fur01] Vladimir Furman. Differential Cryptanalysis of Nimbus. In Mitsuru Matsui, editor, *Fast Software Encryption, 8th International Workshop, FSE 2001 Yokohama, Japan, April 2-4, 2001, Revised Papers*, volume 2355 of *Lecture Notes in Computer Science*, pages 187–195. Springer, 2001. doi:[10.1007/3-540-45473-X_16](https://doi.org/10.1007/3-540-45473-X_16).
- [FWG⁺16] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. MILP-Based Automatic Search Algorithms for Differential and Linear Trails for Speck. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 268–288. Springer, 2016. doi:[10.1007/978-3-662-52993-5_14](https://doi.org/10.1007/978-3-662-52993-5_14).
- [GAED23] Koustabh Ghosh, Parisa Amiri-Eliasi, and Joan Daemen. Multimixer-128: Universal keyed hashing based on integer multiplication. *IACR Trans. Symm. Cryptol.*, 2023(3):1–24, 2023. doi:[10.46586/tosc.v2023.i3.1-24](https://doi.org/10.46586/tosc.v2023.i3.1-24).
- [GLSV15] Vincent Grosso, Gaëtan Leurent, François-Xavier Standaert, and Kerem Varici. LS-designs: Bitslice encryption for efficient masked software implementations. In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 18–37. Springer, Berlin, Heidelberg, March 2015. doi:[10.1007/978-3-662-46706-0_2](https://doi.org/10.1007/978-3-662-46706-0_2).
- [GNNV01] Louis Granboulan, Phong Q. Nguyen, Fabrice Noilhan, and Serge Vaudenay. DFCv2. In Douglas R. Stinson and Stafford E. Tavares, editors, *SAC 2000*, volume 2012 of *LNCS*, pages 57–71. Springer, Berlin, Heidelberg, August 2001. doi:[10.1007/3-540-44983-3_5](https://doi.org/10.1007/3-540-44983-3_5).
- [GPT21] David Gerault, Thomas Peyrin, and Quan Quan Tan. Exploring differential-based distinguishers and forgeries for ASCON. *IACR Trans. Symm. Cryptol.*, 2021(3):102–136, 2021. doi:[10.46586/tosc.v2021.i3.102-136](https://doi.org/10.46586/tosc.v2021.i3.102-136).
- [HDE24] Hosein Hadipour, Patrick Derbez, and Maria Eichlseder. Revisiting Differential-Linear Attacks via a Boomerang Perspective with Application to AES, Ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, Simeck, and SERPENT. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, volume 14923 of *Lecture Notes in Computer Science*, pages 38–72. Springer, 2024. doi:[10.1007/978-3-031-68385-5_2](https://doi.org/10.1007/978-3-031-68385-5_2).
- [HK97] Shai Halevi and Hugo Krawczyk. MMH: Software message authentication in the Gbit/second rates. In Eli Biham, editor, *FSE'97*, volume 1267 of *LNCS*, pages 172–189. Springer, Berlin, Heidelberg, January 1997. doi:[10.1007/BFb0052345](https://doi.org/10.1007/BFb0052345).

- [HLK⁺14] Deukjo Hong, Jung-Keun Lee, Dong-Chan Kim, Daesung Kwon, Kwon Ho Ryu, and Dong-Geon Lee. LEA: A 128-bit block cipher for fast encryption on common processors. In Yongdae Kim, Heejo Lee, and Adrian Perrig, editors, *WISA 13*, volume 8267 of *LNCS*, pages 3–27. Springer, Cham, August 2014. doi:10.1007/978-3-319-05149-9_1.
- [HNE22] Hosein Hadipour, Marcel Nageler, and Maria Eichlseder. Throwing Boomerangs into Feistel Structures Application to CLEFIA, WARP, LBlock, LBlock-s and TWINE. *IACR Trans. Symmetric Cryptol.*, 2022(3):271–302, 2022. doi:10.46586/TOSC.V2022.I3.271-302.
- [HW20] Mingjiang Huang and Liming Wang. Automatic Search for the Linear (Hull) Characteristics of ARX Ciphers: Applied to SPECK, SPARX, Chaskey, and CHAM-64. *Secur. Commun. Networks*, 2020:4898612:1–4898612:14, 2020. doi:10.1155/2020/4898612.
- [IMGM15] Tetsu Iwata, Kazuhiko Minematsu, Jian Guo, and Sumio Morioka. CLOC: Authenticated encryption for short input. In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 149–167. Springer, Berlin, Heidelberg, March 2015. doi:10.1007/978-3-662-46706-0_8.
- [JRS22] Amit Jana, Mostafizar Rahman, and Dhiman Saha. DEEPAND: In-Depth Modeling of Correlated AND Gates for NLFSR-based Lightweight Block Ciphers. *IACR Cryptol. ePrint Arch.*, page 1123, 2022. URL: <https://eprint.iacr.org/2022/1123>.
- [KA24] Emre Karabulut and Aydin Aysu. Masking falcon’s floating-point multiplication in hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):483–508, 2024. URL: <https://doi.org/10.46586/tches.v2024.i4.483-508>, doi:10.46586/TCHES.V2024.I4.483-508.
- [KHL10] Jongsung Kim, Seokhie Hong, and Jongin Lim. Impossible differential cryptanalysis using matrix method. *Discret. Math.*, 310(5):988–1002, 2010. doi:10.1016/J.DISC.2009.10.019.
- [KLS⁺08] Jongsung Kim, Changhoon Lee, Jaechul Sung, Seokhie Hong, Sangjin Lee, and Jongin Lim. Seven new block cipher structures with provable security against differential cryptanalysis. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 91-A(10):3047–3058, 2008. URL: <https://doi.org/10.1093/ietfec/e91-a.10.3047>, doi:10.1093/IETFEC/E91-A.10.3047.
- [KN10] Dmitry Khovratovich and Ivica Nikolic. Rotational Cryptanalysis of ARX. In Seokhie Hong and Tetsu Iwata, editors, *Fast Software Encryption, 17th International Workshop, FSE 2010, Seoul, Korea, February 7-10, 2010, Revised Selected Papers*, volume 6147 of *Lecture Notes in Computer Science*, pages 333–346. Springer, 2010. doi:10.1007/978-3-642-13858-4_19.
- [Knu94] Lars R. Knudsen. Truncated and Higher Order Differentials. In Bart Preneel, editor, *Fast Software Encryption: Second International Workshop. Leuven, Belgium, 14-16 December 1994, Proceedings*, volume 1008 of *Lecture Notes in Computer Science*, pages 196–211. Springer, 1994. doi:10.1007/3-540-60590-8_16.
- [Knu02] Lars R. Knudsen. The Security of Feistel Ciphers with Six Rounds or Less. *J. Cryptol.*, 15(3):207–222, 2002. doi:10.1007/S00145-002-9839-Y.

- [KR99] Lars R. Knudsen and Vincent Rijmen. On the decorrelated fast cipher (dfc) and its theory. In Lars R. Knudsen, editor, *FSE'99*, volume 1636 of *LNCS*, pages 81–94. Springer, Berlin, Heidelberg, March 1999. doi:[10.1007/3-540-48519-8_7](https://doi.org/10.1007/3-540-48519-8_7).
- [KW02] Lars R. Knudsen and David A. Wagner. Integral Cryptanalysis. In Joan Daemen and Vincent Rijmen, editors, *Fast Software Encryption, 9th International Workshop, FSE 2002, Leuven, Belgium, February 4-6, 2002, Revised Papers*, volume 2365 of *Lecture Notes in Computer Science*, pages 112–127. Springer, 2002. doi:[10.1007/3-540-45661-9_9](https://doi.org/10.1007/3-540-45661-9_9).
- [Lai94] Xuejia Lai. Higher order derivatives and differential cryptanalysis. *Communications and Cryptography: Two Sides of One Tapestry*, pages 227–233, 1994. doi:[10.1007/978-1-4615-2694-0_23](https://doi.org/10.1007/978-1-4615-2694-0_23).
- [LFW⁺16] Yu Liu, Kai Fu, Wei Wang, Ling Sun, and Meiqin Wang. Linear cryptanalysis of reduced-round SPECK. *Inf. Process. Lett.*, 116(3):259–266, 2016. URL: <https://doi.org/10.1016/j.ipl.2015.11.005>, doi:[10.1016/J.IPL.2015.11.005](https://doi.org/10.1016/J.IPL.2015.11.005).
- [LH94] Susan K. Langford and Martin E. Hellman. Differential-Linear Cryptanalysis. In Yvo Desmedt, editor, *Advances in Cryptology - CRYPTO '94, 14th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1994, Proceedings*, volume 839 of *Lecture Notes in Computer Science*, pages 17–25. Springer, 1994. doi:[10.1007/3-540-48658-5_3](https://doi.org/10.1007/3-540-48658-5_3).
- [LM91] Xuejia Lai and James L. Massey. A proposal for a new block encryption standard. In Ivan Damgård, editor, *EUROCRYPT'90*, volume 473 of *LNCS*, pages 389–404. Springer, Berlin, Heidelberg, May 1991. doi:[10.1007/3-540-46877-3_35](https://doi.org/10.1007/3-540-46877-3_35).
- [Mac00] Alexis Warner Machado. The Nimbus cipher: A proposal for NESSIE. . NESSIE Proposal, 2000.
- [Mat97] Mitsuru Matsui. New block encryption algorithm MISTY. In Eli Biham, editor, *FSE'97*, volume 1267 of *LNCS*, pages 54–68. Springer, Berlin, Heidelberg, January 1997. doi:[10.1007/BFb0052334](https://doi.org/10.1007/BFb0052334).
- [MMV⁺14] Nicky Mouha, Bart Mennink, Anthony Van Herrewege, Dai Watanabe, Bart Preneel, and Ingrid Verbauwhede. Chaskey: An efficient MAC algorithm for 32-bit microcontrollers. In Antoine Joux and Amr M. Youssef, editors, *SAC 2014*, volume 8781 of *LNCS*, pages 306–323. Springer, Cham, August 2014. doi:[10.1007/978-3-319-13051-4_19](https://doi.org/10.1007/978-3-319-13051-4_19).
- [MNSV97] David M'Raihi, David Naccache, Jacques Stern, and Serge Vaudenay. XMx: A Firmware-Oriented Block Cipher Based on Modular Multiplications. In Eli Biham, editor, *Fast Software Encryption, 4th International Workshop, FSE '97, Haifa, Israel, January 20-22, 1997, Proceedings*, volume 1267 of *Lecture Notes in Computer Science*, pages 166–171. Springer, 1997. doi:[10.1007/BFb0052344](https://doi.org/10.1007/BFb0052344).
- [Mur11] Sean Murphy. The Return of the Cryptographic Boomerang. *IEEE Trans. Inf. Theory*, 57(4):2517–2521, 2011. doi:[10.1109/TIT.2011.2111091](https://doi.org/10.1109/TIT.2011.2111091).
- [MV04] David A. McGrew and John Viega. The security and performance of the Galois/counter mode (GCM) of operation. In Anne Canteaut and Kapalee

- Viswanathan, editors, *INDOCRYPT 2004*, volume 3348 of *LNCS*, pages 343–355. Springer, Berlin, Heidelberg, December 2004. doi:[10.1007/978-3-540-30556-9_27](https://doi.org/10.1007/978-3-540-30556-9_27).
- [NK93] Kaisa Nyberg and Lars R. Knudsen. Provable security against differential cryptanalysis (rump session). In Ernest F. Brickell, editor, *CRYPTO'92*, volume 740 of *LNCS*, pages 566–574. Springer, Berlin, Heidelberg, August 1993. doi:[10.1007/3-540-48071-4_41](https://doi.org/10.1007/3-540-48071-4_41).
- [NK95] Kaisa Nyberg and Lars R. Knudsen. Provable security against a differential attack. *Journal of Cryptology*, 8(1):27–37, December 1995. doi:[10.1007/BF00204800](https://doi.org/10.1007/BF00204800).
- [Noi99] Fabrice Noilhan. Software optimization of decorrelation module. In Howard M. Heys and Carlisle M. Adams, editors, *SAC 1999*, volume 1758 of *LNCS*, pages 175–183. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-46513-8_13](https://doi.org/10.1007/3-540-46513-8_13).
- [NR12] Long Hoang Nguyen and A. W. Roscoe. Short-output universal hash functions and their use in fast and secure data authentication. In Anne Canteaut, editor, *FSE 2012*, volume 7549 of *LNCS*, pages 326–345. Springer, Berlin, Heidelberg, March 2012. doi:[10.1007/978-3-642-34047-5_19](https://doi.org/10.1007/978-3-642-34047-5_19).
- [Nyb95] Kaisa Nyberg. Linear approximation of block ciphers (rump session). In Alfredo De Santis, editor, *EUROCRYPT'94*, volume 950 of *LNCS*, pages 439–444. Springer, Berlin, Heidelberg, May 1995. doi:[10.1007/BFb0053460](https://doi.org/10.1007/BFb0053460).
- [Pat08] Jacques Patarin. Generic Attacks on Feistel Schemes. *IACR Cryptol. ePrint Arch.*, page 36, 2008. URL: <http://eprint.iacr.org/2008/036>.
- [Por18] Thomas Pornin. Constant-Time Mul, 2018. URL: <https://bearssl.org/ctmul.html>.
- [PV98] Guillaume Poupard and Serge Vaudenay. Decorrelated fast cipher: An AES candidate well suited for low cost smart card applications. In Jean-Jacques Quisquater and Bruce Schneier, editors, *Smart Card Research and Applications, This International Conference, CARDIS '98, Louvain-la-Neuve, Belgium, September 14-16, 1998, Proceedings*, volume 1820 of *Lecture Notes in Computer Science*, pages 254–264. Springer, 1998. doi:[10.1007/10721064_24](https://doi.org/10.1007/10721064_24).
- [RRY98] Ronald L. Rivest, Matthew J. B. Robshaw, and Yiqun Lisa Yin. The RC6 Block Cipher, 1998.
- [Scr01] Beale Screamer. Microsoft's Digital Rights Management Scheme—Technical Details. <http://cryptome.org/ms-drm.htm>, October 2001. Accessed: 2025-07-03.
- [SHY16] Ling Song, Zhangjie Huang, and Qianqian Yang. Automatic Differential Analysis of ARX Block Ciphers with Application to SPECK and LEA. In Joseph K. Liu and Ron Steinfield, editors, *Information Security and Privacy - 21st Australasian Conference, ACISP 2016, Melbourne, VIC, Australia, July 4-6, 2016, Proceedings, Part II*, volume 9723 of *Lecture Notes in Computer Science*, pages 379–394. Springer, 2016. doi:[10.1007/978-3-319-40367-0_24](https://doi.org/10.1007/978-3-319-40367-0_24).

- [SMMK13] Tomoyasu Suzaki, Kazuhiko Minematsu, Sumio Morioka, and Eita Kobayashi. TWINE : A lightweight block cipher for multiple platforms. In Lars R. Knudsen and Huapeng Wu, editors, *SAC 2012*, volume 7707 of *LNCS*, pages 339–354. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-35999-6_22.
- [spe21] simon-speck-supercop, 2021. URL: https://github.com/nsacyber/simon-speck-supercop/tree/master/crypto_stream.
- [SPGQ06] François-Xavier Standaert, Gilles Piret, Neil Gershenfeld, and Jean-Jacques Quisquater. SEA: A scalable encryption algorithm for small embedded applications. In Josep Domingo-Ferrer, Joachim Posegga, and Daniel Schreckling, editors, *Smart Card Research and Advanced Applications, 7th IFIP WG 8.8/11.2 International Conference, CARDIS 2006, Tarragona, Spain, April 19-21, 2006, Proceedings*, volume 3928 of *Lecture Notes in Computer Science*, pages 222–236. Springer, 2006. doi:10.1007/11733447_16.
- [SSS⁺20] Dhiman Saha, Yu Sasaki, Danping Shi, Ferdinand Sibleyras, Siwei Sun, and Yingjie Zhang. On the Security Margin of TinyJAMBU with Refined Differential and Linear Cryptanalysis. *IACR Trans. Symmetric Cryptol.*, 2020(3):152–174, 2020. doi:10.13154/TOSC.V2020.I3.152-174.
- [Tho15] Mikkel Thorup. High Speed Hashing for Integers and Strings. *CoRR*, abs/1504.06804, 2015.
- [Tod15] Yosuke Todo. Structural Evaluation by Generalized Integral Property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015. doi:10.1007/978-3-662-46800-5_12.
- [TP09] Joana Treger and Jacques Patarin. Generic Attacks on Feistel Networks with Internal Permutations. In Bart Preneel, editor, *Progress in Cryptology - AFRICACRYPT 2009, Second International Conference on Cryptology in Africa, Gammarth, Tunisia, June 21-25, 2009. Proceedings*, volume 5580 of *Lecture Notes in Computer Science*, pages 41–59. Springer, 2009. doi:10.1007/978-3-642-02384-2_4.
- [TSL11] Xuehai Tang, Bing Sun, Ruilin Li, and Chao Li. Impossible differential cryptanalysis of 13-round CLEFIA-128. *J. Syst. Softw.*, 84(7):1191–1196, 2011. doi:10.1016/J.JSS.2011.02.024.
- [Vau98] Serge Vaudenay. Provable security for block ciphers by decorrelation. In Michel Morvan, Christoph Meinel, and Daniel Krob, editors, *STACS 98, 15th Annual Symposium on Theoretical Aspects of Computer Science, Paris, France, February 25-27, 1998, Proceedings*, volume 1373 of *Lecture Notes in Computer Science*, pages 249–275. Springer, 1998. URL: <https://doi.org/10.1007/BFb0028566>, doi:10.1007/BFB0028566.
- [Vau99a] Serge Vaudenay. Adaptive-attack norm for decorrelation and superpseudorandomness. In Howard M. Heys and Carlisle M. Adams, editors, *SAC 1999*, volume 1758 of *LNCS*, pages 49–61. Springer, Berlin, Heidelberg, August 1999. doi:10.1007/3-540-46513-8_4.

- [Vau99b] Serge Vaudenay. Feistel ciphers with L_2 -decorrelation. In Stafford E. Tavares and Henk Meijer, editors, *SAC 1998*, volume 1556 of *LNCS*, pages 1–14. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-48892-8_1](https://doi.org/10.1007/3-540-48892-8_1).
- [Vau03] Serge Vaudenay. Decorrelation: A theory for block cipher security. *Journal of Cryptology*, 16(4):249–286, September 2003. doi:[10.1007/s00145-003-0220-6](https://doi.org/10.1007/s00145-003-0220-6).
- [Wag99] David A. Wagner. The Boomerang Attack. In Lars R. Knudsen, editor, *Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24-26, 1999, Proceedings*, volume 1636 of *Lecture Notes in Computer Science*, pages 156–170. Springer, 1999. doi:[10.1007/3-540-48519-8_12](https://doi.org/10.1007/3-540-48519-8_12).
- [WN95] David J. Wheeler and Roger M. Needham. TEA, a tiny encryption algorithm. In Bart Preneel, editor, *FSE'94*, volume 1008 of *LNCS*, pages 363–366. Springer, Berlin, Heidelberg, December 1995. doi:[10.1007/3-540-60590-8_29](https://doi.org/10.1007/3-540-60590-8_29).
- [WP19] Haoyang Wang and Thomas Peyrin. Boomerang Switch in Multiple Rounds. Application to AES Variants and Deoxys. *IACR Trans. Symmetric Cryptol.*, 2019(1):142–169, 2019. doi:[10.13154/TOSC.V2019.I1.142-169](https://doi.org/10.13154/TOSC.V2019.I1.142-169).
- [WWS23] Dachao Wang, Baocang Wang, and Siwei Sun. SAT-aided Automatic Search of Boomerang Distinguishers for ARX Ciphers. *IACR Trans. Symmetric Cryptol.*, 2023(1):152–191, 2023. doi:[10.46586/TOSC.V2023.I1.152-191](https://doi.org/10.46586/TOSC.V2023.I1.152-191).
- [WZ11] Wenling Wu and Lei Zhang. LBlock: A lightweight block cipher. In Javier Lopez and Gene Tsudik, editors, *ACNS 2011*, volume 6715 of *LNCS*, pages 327–344. Springer, Berlin, Heidelberg, June 2011. doi:[10.1007/978-3-642-21554-4_19](https://doi.org/10.1007/978-3-642-21554-4_19).
- [ZBL⁺14] Wentao Zhang, Zhenzhen Bao, Dongdai Lin, Vincent Rijmen, Bohan Yang, and Ingrid Verbauwhede. RECTANGLE: A bit-slice ultra-lightweight block cipher suitable for multiple platforms. *IACR Cryptol. ePrint Arch.*, page 84, 2014.